

Next Week:

Ethics, privacy & other concerns

First, Oksana Dorofeeva, who's a researcher at Department of Political Science, will share her expertise on the “social and ethical dimensions of machine learning”.

Afterwards, I'll give a somewhat shorter presentation on technical solutions to some of the problems highlighted by Oksana.



GAI at exams

If you use part of a text or another output generated by a GAI application in your exam project without changing it, **you must cite it** in the same way you cite quotations from other secondary sources (Read the rules on citations here).

If you use GAI in your exam project, **you must submit a declaration** that contains the following: 1) confirmation you used GAI, 2) the name of the GAI applications you used (ChatGPT, Copilot, Bing etc. and 3) an explanation of how you used the applications.

You must submit the declaration as an attachment when you submit your exam project if you use GAI. To write your declaration, fill out the AU template you'll find here. Unless you refer to your use of GAI in the methodology section of your paper, your declaration and your description of how you used GAI in the exam project will not be part of your grade.

If you don't follow these rules, it counts as cheating.

Rest of course

No exercises

-> But your TA's are still available to answer questions about your project / torch / LUMI (possibly). Coordinate with them how get in touch (exercise sessions or just by mail, for instance)

One 'bonus' lecture:

21/11: Privacy, Bias and other ethical concerns

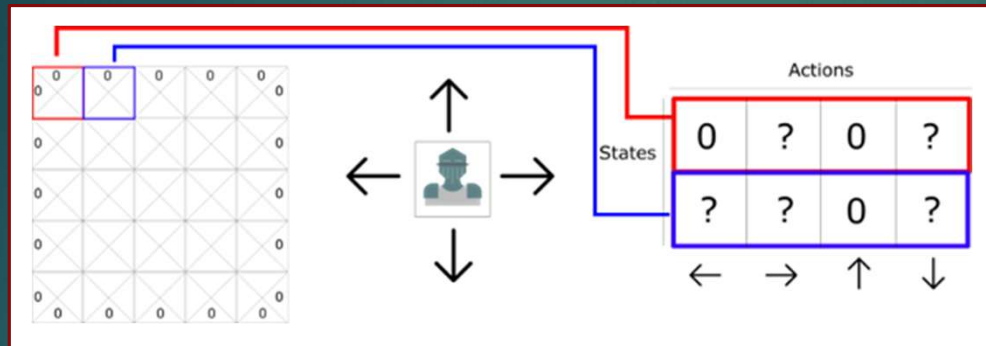
Remember to evaluate the course!

Network design and other hyperparameters

DEEP LEARNING LECTURE 11

Last week – Reinforcement learning:

Q-learning:



Deep Q-learning:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{U}(D)} \left[\left(\underbrace{r}_{\text{Reward}} + \underbrace{\gamma}_{\text{Discount rate}} \underbrace{\max_{a'} Q(s', a'; \theta_i^-)}_{\text{Max Q value of next state } s' \text{ and possible actions } a' \text{ at that state}} - \underbrace{Q(s,a; \theta_i)}_{\text{Current Q value}} \right)^2 \right]$$

Sample from replay memory
 The network parameters used to compute the target at iteration j
 The parameters of the Q-network at iteration i

Dueling Deep Q:

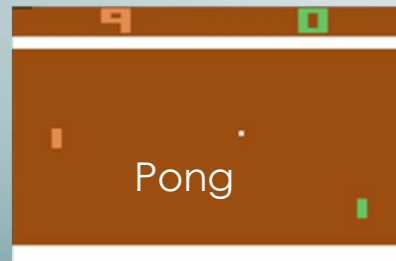
$$Q(s, a) = \underbrace{A(s, a)}_{\text{Action value}} + \underbrace{V(s)}_{\text{State value}}$$

Quiz!

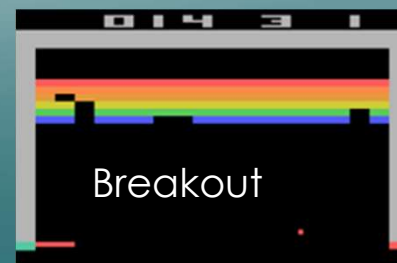
Can you name all these games without checking?

Who beat them?

6



Google
deepmind



Google
deepmind

Assignment 2 & pitch comments



Art & Science of deep learning

8

Part of what makes deep learning hard to master, is the issue of picking the optimal hyperparameters. While there are good practices to follow, there can still be a bit of an art to it.

Today we will talk about:

- Train/val/test split
- Validation and training loss
- Hyperparameter search
- Network design search
- Loss landscapes

Hyperparameters:

In this talk,
'hyperparameter'
covers any part of the
learning process
which is not adjusted
through gradients.

Batch size

Learning rate

Regularization
(batch norm,
drop out, L1,
L2,...)

Loss function

Optimizer

Data
augmentation

Data set

Network design
/ dimensions

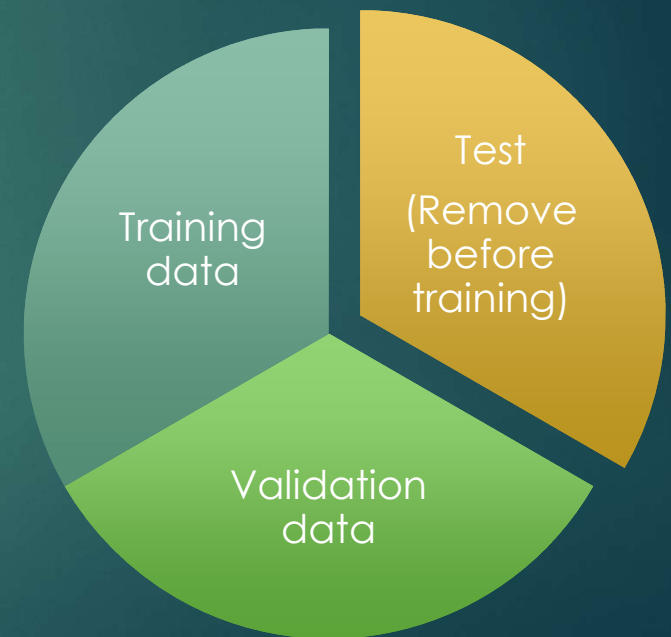
Training, validation, test

10

Quick reminder:

Split into both **training**, **validation** and **test** data. During hyper parameter search, you are going to train repeatedly on the data, and evaluate your decisions on the validation data. During this process you are bound to overfit to the validation data.

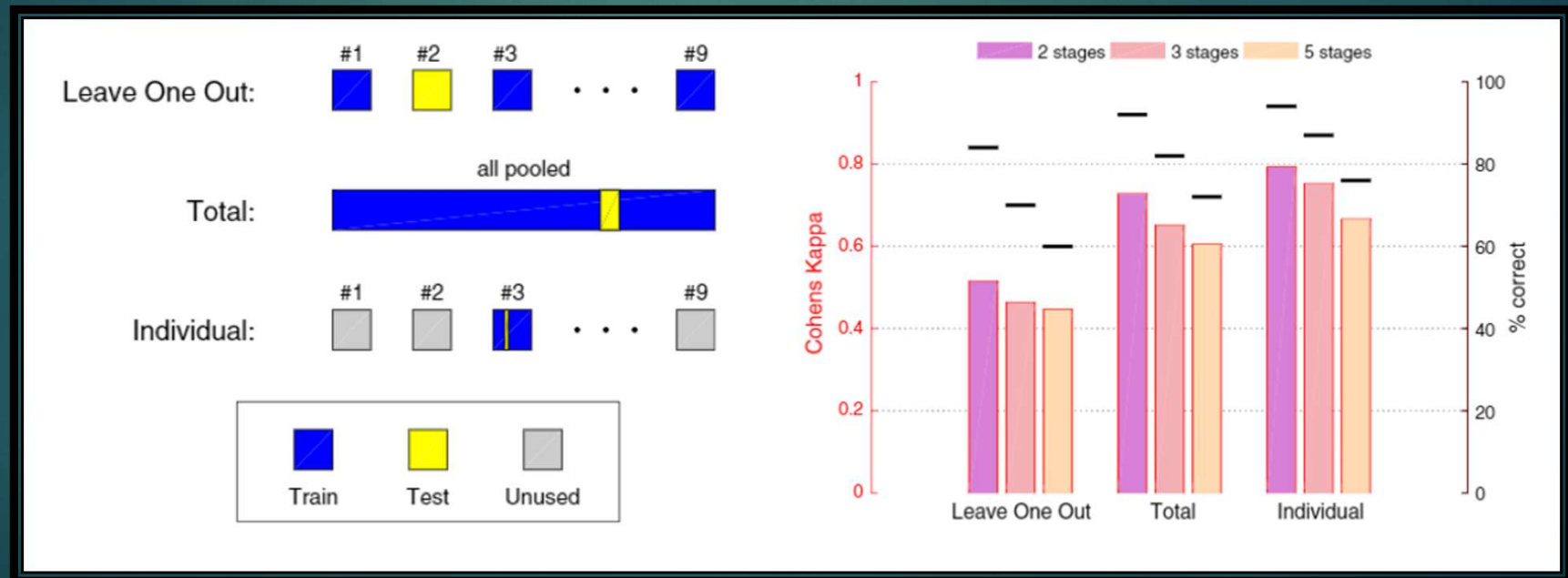
Ideally, don't run on the test data until you're ready to publish / present.



When splitting:

When you split, make sure to not accidentally leave information in the test set that you will not have access to 'in production'.

In the figure below, we tested the effect of various ways to split the sleep data from 9 different people. 'Leave-one-out' is the correct split, and we see that the alternatives significantly overestimate model performance.



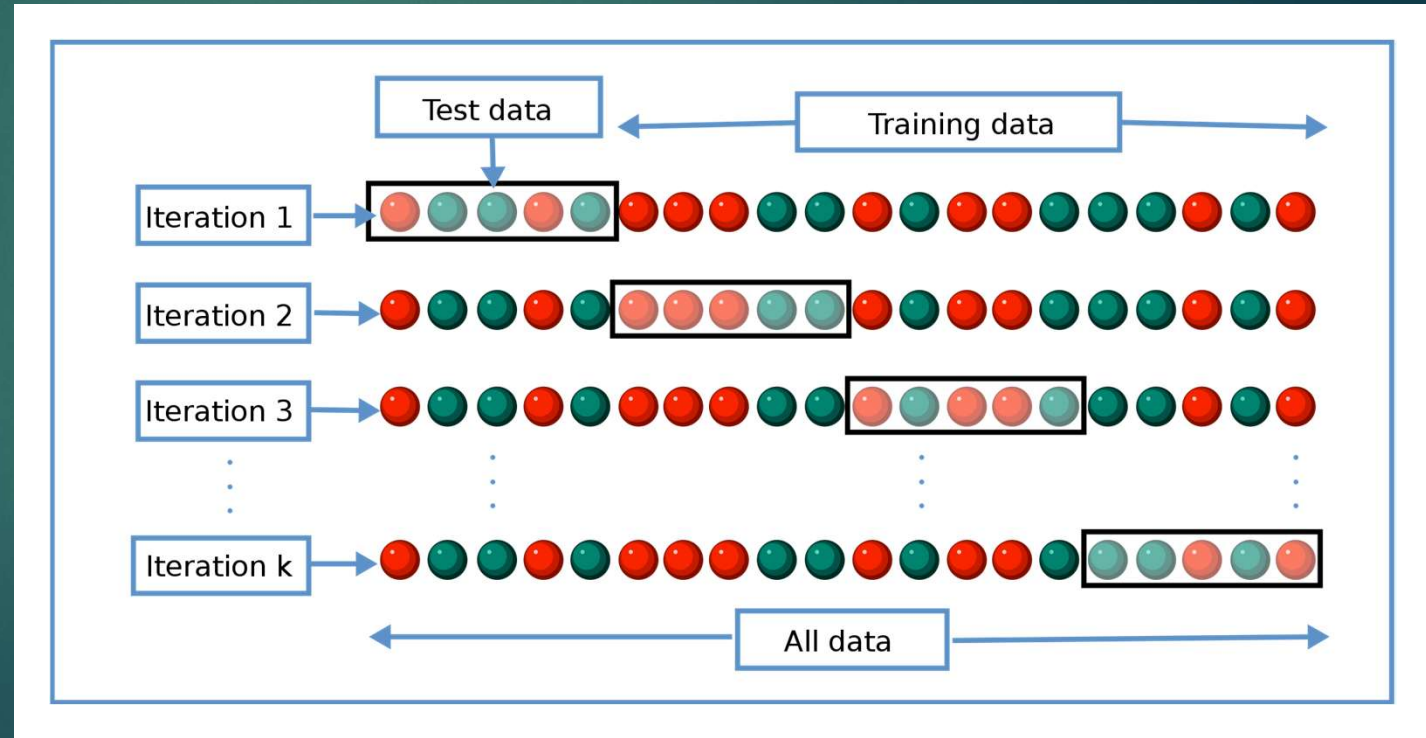
<https://pubmed.ncbi.nlm.nih.gov/28927417/>

Cross validation

Remember that we can use cross validation, where the full training data set is split into K parts, which take turn being validation data.

This means more validation data, and more trials of the same hyperparameter settings.

This, in turn, gives a better estimate of how well this hyperparameter set performs.



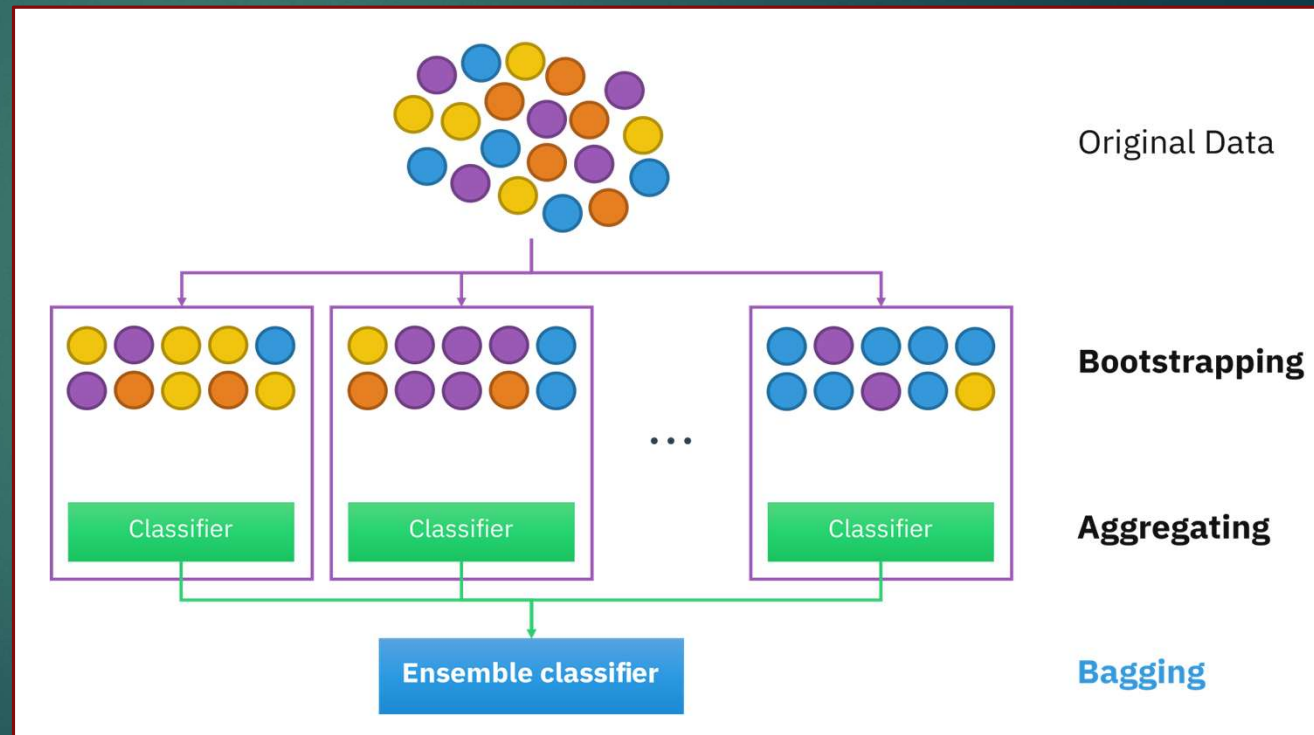
Bootstrapping?

An extension of this is the more general **boot strapping**, where the original training data is resampled repeatedly, to get a high number of similar training data sets.

The benefit of this is that we can better separate the variances from data and training procedure.

The downside is the added computation time...

My impression is that bootstrapping is not usually done for deep learning.



Train/test quiz:



14

We are designing a study to determine people's favourite pizza toppings, relative to their **height, gender, favourite color and religion**.

Relative to good test/train practices, what could be good practices in our study?

1. Have separate people collect data and do the analysis.
2. Run the analysis during data collection, to ensure sufficient data is collected.
3. Randomly select one third of the data, ask your neighbour to hide it from you until you're done fitting a model.
4. Do two rounds: first ask a lot of students (easy to get to), fit a model to that, and then test it on a second round of data, collected from non-students (to avoid overlap).
5. Take 10 percent of the recordings and burn them, that way we can't be influenced by them.
6. These are all bad suggestions



Thoughts:

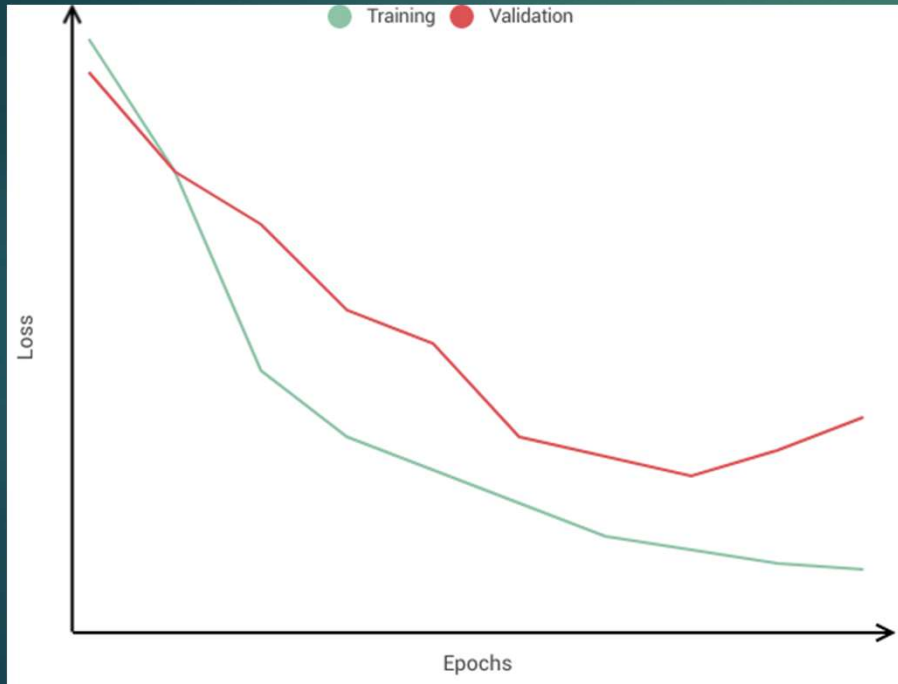


1. Splitting the workload is excellent. Makes it much easier to not get influenced by test data when you're fitting models (because you haven't seen all the data).
2. Be VERY careful about letting your analysis guide your measurement setup. It's really easy to accidentally fit to your test data that way.
3. This is just train/test splitting with a weird extra step. Good idea.
4. Don't do this. Those two populations will be different - and you're not even asking the participants if they are students or not, so the analysis will probably not pick up on it.
5. If we can't actually test on the data at the end, it's not really test data.

Loss or error graphs

16

An important tool in evaluating machine learning models is the loss curve:



Look for:

- Is training loss sufficiently low? You should expect to see some overfitting on the training data.
- Does validation loss converge? If not, adjust number of epochs or learning rate / optimizer
- Is validation loss too high? Consider more regularization if so.
- What would be your own performance on this data? Does the model approach that?

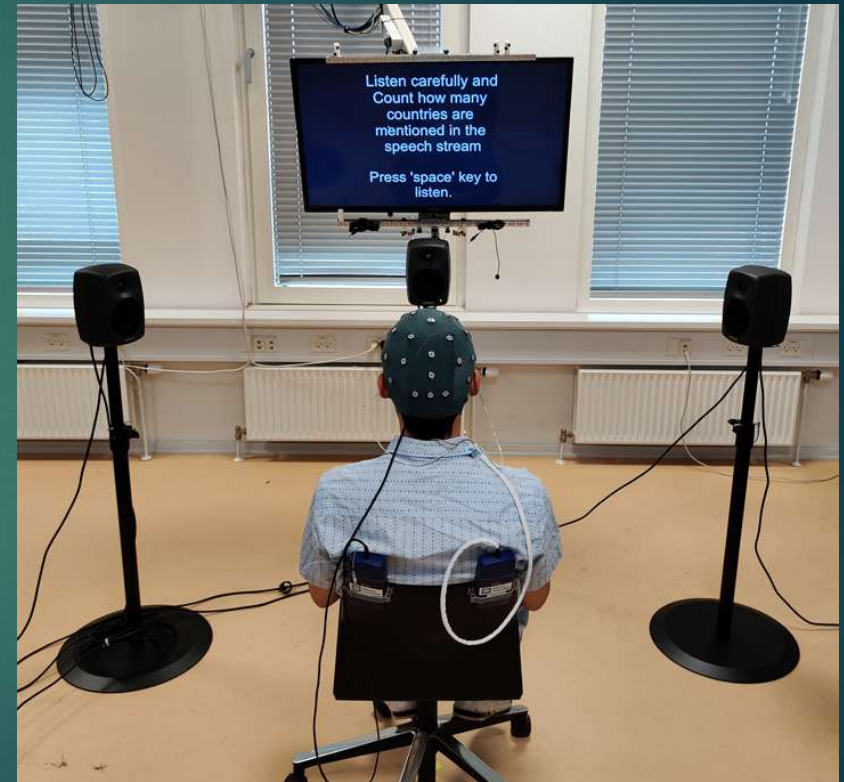
(Please note that it is not always the case that validation loss grows at the end. It's just something that *can* happen.)

Cautionary tale:

A PhD student is investigating a new technique for decoding auditory attention, for use in future smart hearing aids.

In their experiment, different sound streams were played from different speakers, and a machine learning model was trained to guess which stream was being attended, based on the EEG recording.

In preparing the data set, the phd student made two streams for each sub-experiment: a 'stream 1' and a 'stream 2'.





Cautionary tale:

To properly control for correlations between stimuli and targets, all streams were played twice: one in which they were attended, and one where they weren't attended.

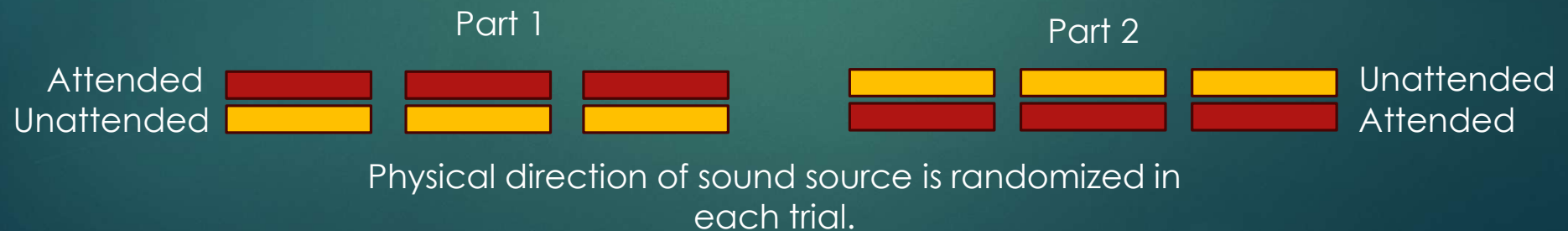
To make everything easier, the student simply made two portions of the experiment. In the first half, 'stream 1' was attended, and in the second half, 'stream 2' was attended. In this way, everything was treated equally. To make extra sure, the actual direction (left/right) from where the attended sound was played was randomized.

Cautionary tale

However, when the data was actually analyzed, it turned out that the model was performing much too well. It turned out that it could actually guess which sound was being listened to (stream 1 or stream 2) without actually being given any information about the sound.

....

So what happened?



Cautionary tale:

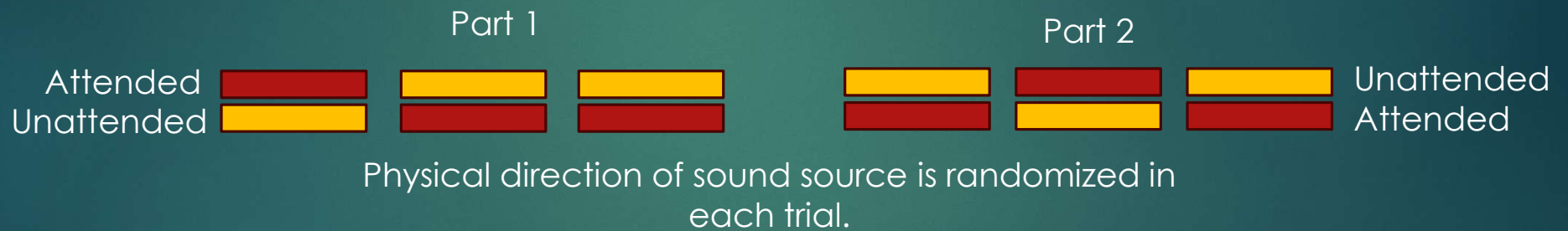
When the model had to guess which sound was being attended, they were presented in a matrix: one row for stream 1, one row for stream 2.

However, since stream 1 was always attended in the first half, and stream 2 always in the second half, if there was anything in the experiment which changed with time, the model would simply have to guess 'early' or 'late', and then it would know which row to pick.

In an experiment like this, things such as subject fatigue or electrode connection quality will in fact depend on time...

Solution:

In this case, we removed the relationship between data representation and stream index:



The result was much more reasonable performance, and all-round happiness 😊

Grid search variations

A common way to scan for good hyperparameter values is a 'grid search', in which multiple parameters are varied simultaneously.

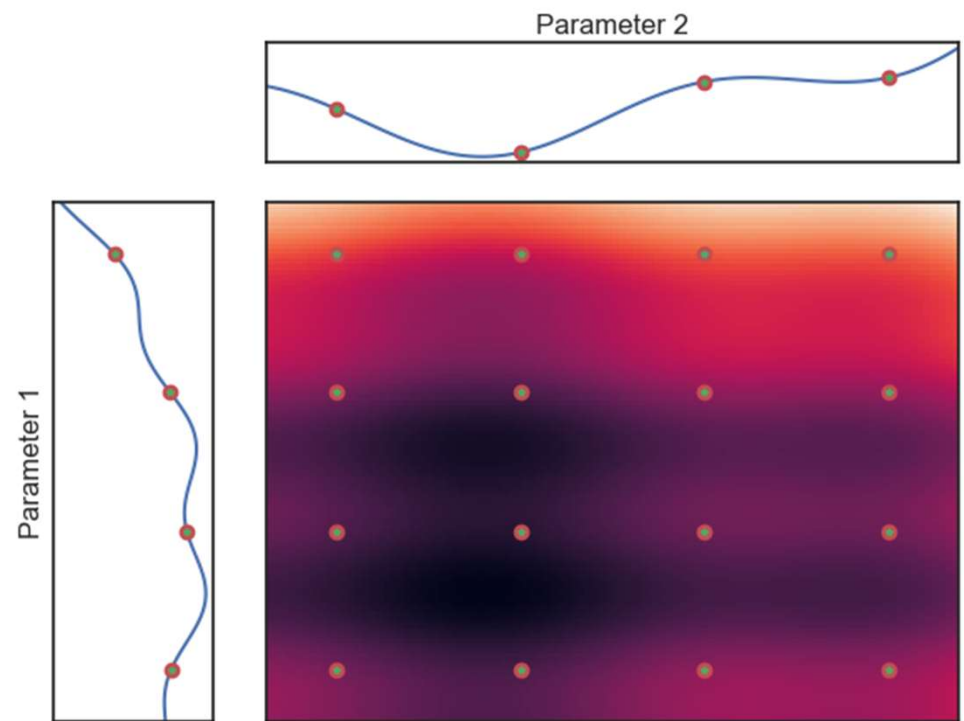
This is done to allow for the possibility that parameters are correlated.

However, it can be a good idea to also allow for the possibility that they are not.

A conventional grid search has very little information about the individual parameter space.

Standard grid

22



NB: the plot above is stylized – the 1D values should not be exactly on the blue lines.

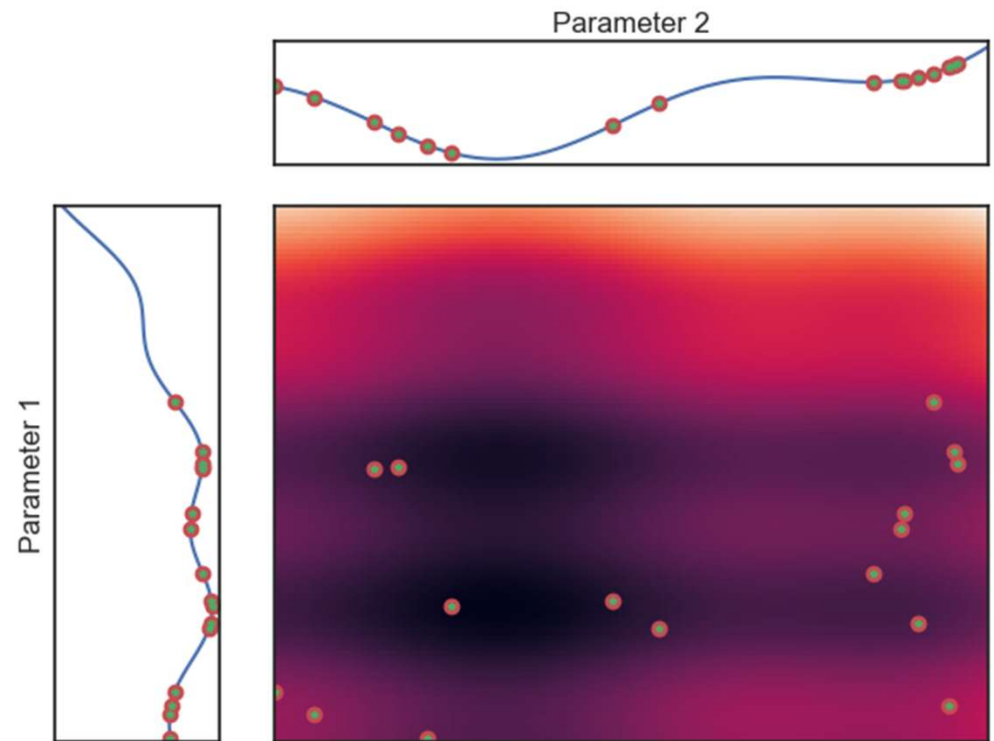
Grid search variations

A simple remedy to this can be a random search. In this case no two points are likely to have the same value for any parameter, and a more detailed picture of each dependence is possible.

However, the uneven spread is a clear downside.

Random search

23



NB: the plot above is stylized – the 1D values should not be exactly on the blue lines.

Grid search variations

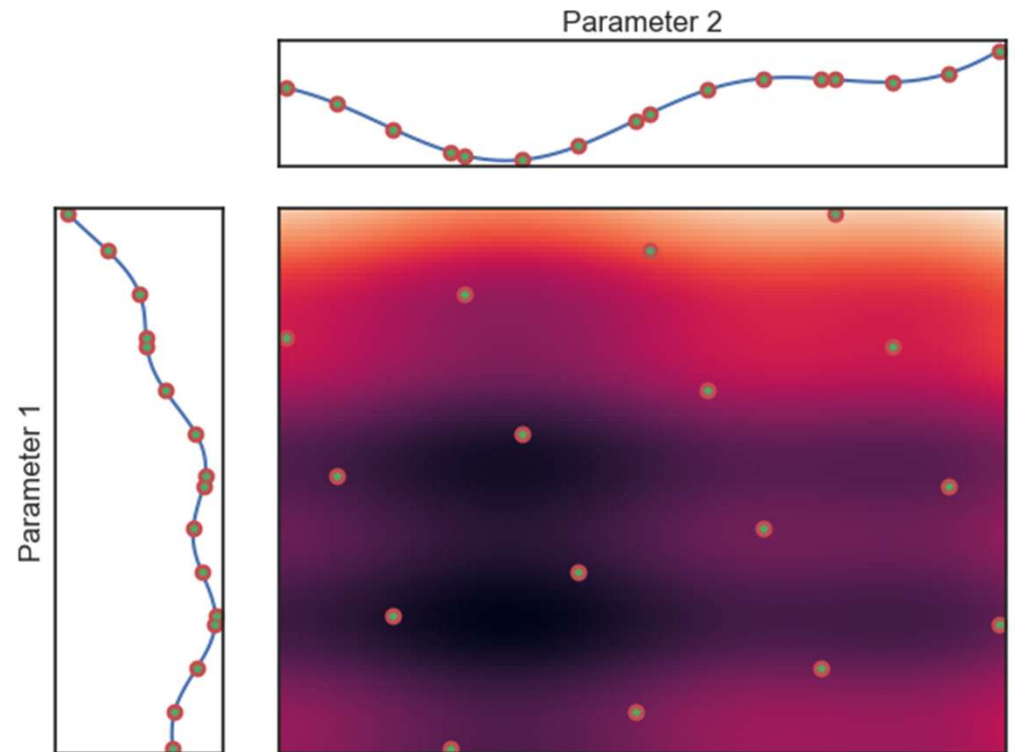
Another attempted upgrade to the grid is the rotated grid search.

This combines the advantages of both no repetitions, as well as a much more even coverage of the joint parameter space.

The main drawback is probably that it requires some extra coding, including some thoughts about rotating in n -dim...

Rotated grid

24



NB: the plot above is stylized – the 1D values should not be exactly on the blue lines.

Grid search variations

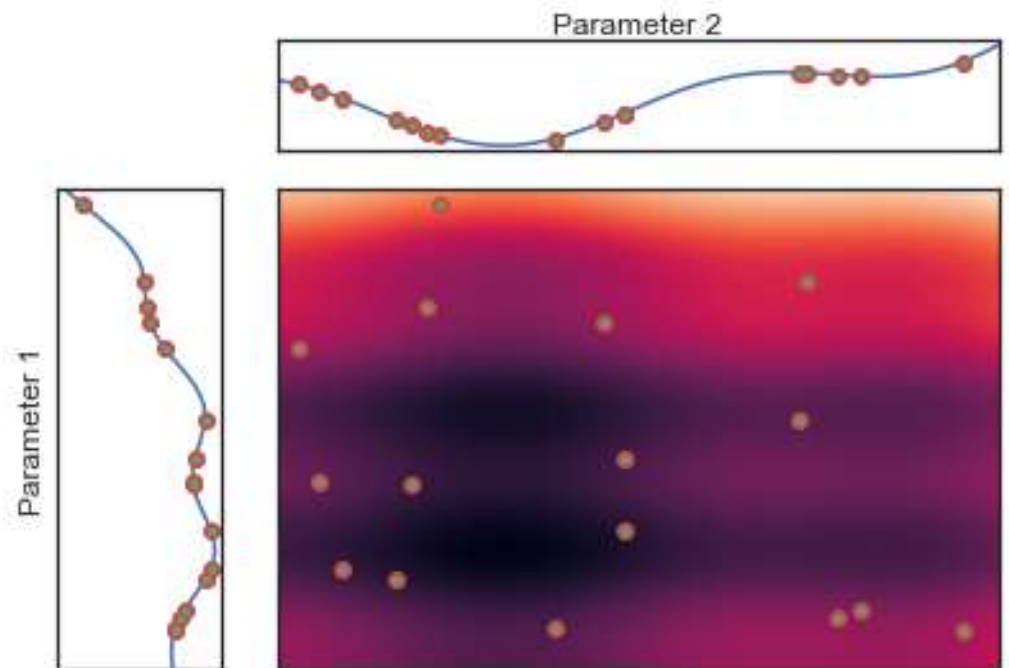
Finally, there is the noisy grid search. This is a compromise between the ease of grids and random, and the smart coverage from the the rotated grid.

It's simply a grid search where the values have a bit of noise added.

(probably my favourite)

Noisy grid

25



NB: the plot above is stylized – the 1D values should not be exactly on the blue lines.

Actual search algorithm

We can also use an actual search algorithm to update our parameters.

I favour the ray.tune library, but there are multiple candidates.

See the lumi-guide for an example of using Ray.tune.



Scikit-optimize



RAY





Break

27

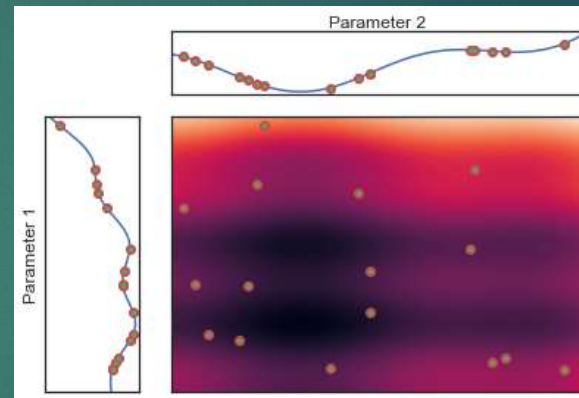
Summing up

28

Data splitting



Search strategies



Data splitting question:

In a project, researchers are lacking training data.

They come up with some sensible data augmentation strategies, which give them 20x as much data as before.

They are now faced with a few options:

- a. Mix original and synthetic data before splitting into train and validation (to ensure no accidental biases)
- b. Split, then augment.
- c. Use all the synthetic data as training, and the original as validation (gives optimally representative validation data)

Bias vs. variance

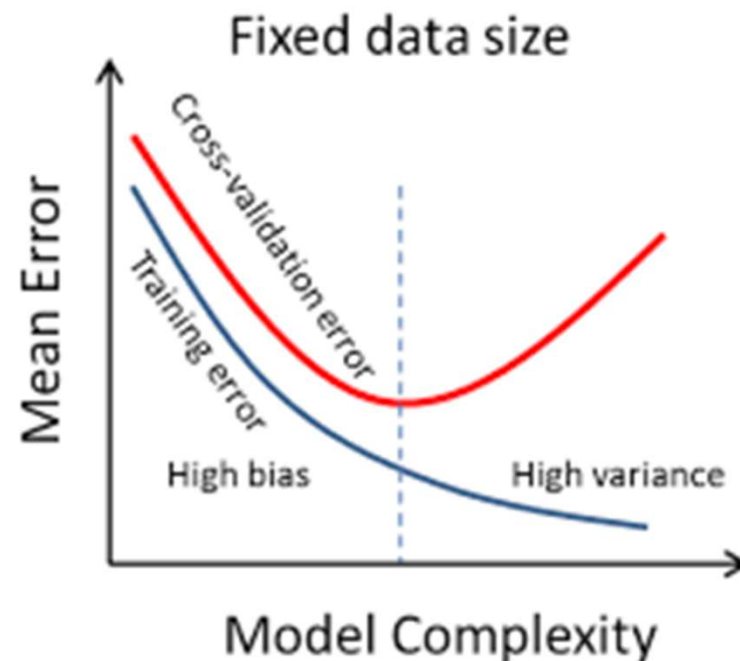
30

A useful way to consider loss curves is as a function of model complexity.

For machine learning models in general, it is usually the case that

- *Simple* models have bias towards specific outputs, and low dependence on initial conditions (variance)
- *Complex* models have little bias, but they overfit to the inputs and initial conditions.

In the sweet spot, we minimize both bias and variance.

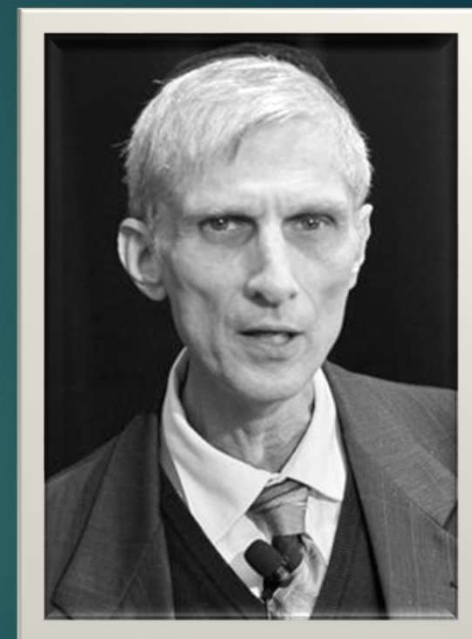


Leslie N. Smith:

In his paper, *'A disciplined approach to neural network hyper-parameters: Part 1'*, Leslie N. Smith shares a wealth of personal insights and experiences about efficient ways to tune hyper parameters.

We should not take Leslie's insights as mathematical truths (they are only anecdotes), however, they are probably a good starting point for you to form your own understanding.

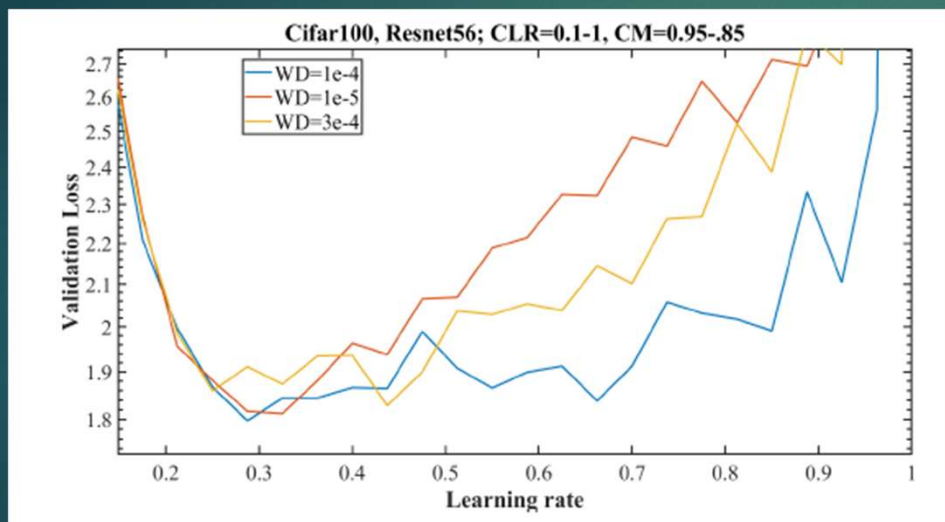
(and no, he never made a part 2)



Mr. Smith himself

Source: <https://arxiv.org/abs/1803.09820>

1: When searching, make sure to use the full range

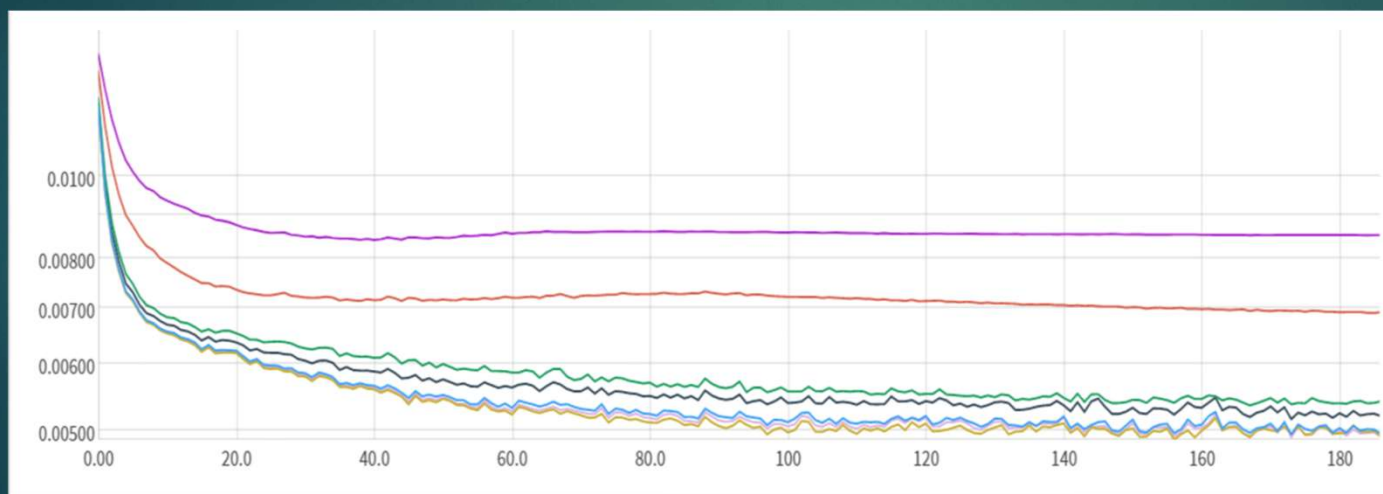


For most settings, there is a 'region of trainability', and outside of this, the loss diverges. Make sure you have found these points of divergence in your search.

2: Models that end low often drop quickly

This one I actually went and did a quick test of with my own favourite model. Here I vary the 'weight decay' of the adam optimizer.

Below is shown validation loss as a function of epoch:

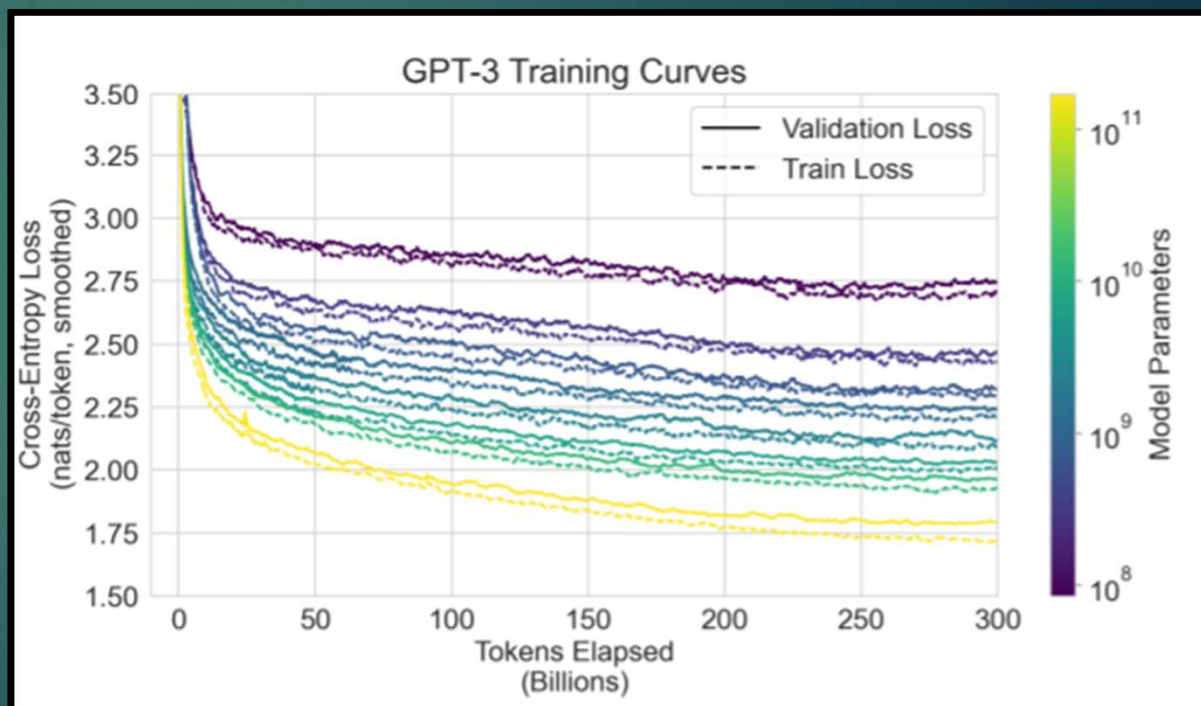


Somehow, it works?

2: Models that end low often drop quickly

I stole this plot from the paper announcing GPT-3. Interestingly, it seems to also hold for their huge models.

It is exceedingly odd.



Source: <https://arxiv.org/abs/2005.14165>

.... Within reason

I don't think this is a law of nature, though it is peculiar that it is even a thing. From the small tests I have made, it seems that if the general architecture is the same, as well as the data set and optimizer, then this behavior is quite normal.

Even if it is not a law of nature, it seems so useful that you could still rely on it for the search process, and then only do 'long training' occasionally, as reality checks.

35



3: A large learning rate is a form of regularization

In general, different forms of regularization overlap, and if you increase one, expect to have to decrease others:

- ▶ Dropout
- ▶ Batch normalization
- ▶ Batch size (inverse effect...)
- ▶ L1 & L2 regularization
- ▶ Learning rate

Since large learning rates may converge quicker, Leslie Smith suggests using as large a learning rate as possible, and then tone down the use of other forms of regularization.

4: Learning rate schedulers

- If the learning rate is too large, we may overshoot the minimum, and it can be hard to combine learning across batches (so, the 'stochastic' in SGD becomes too dominant).
- If the learning rate is too small, learning will take forever, and we may get stuck in local minima (too little stochasticity).

Some researchers report that the best choice is a varying learning rate, using a learning rate scheduler.



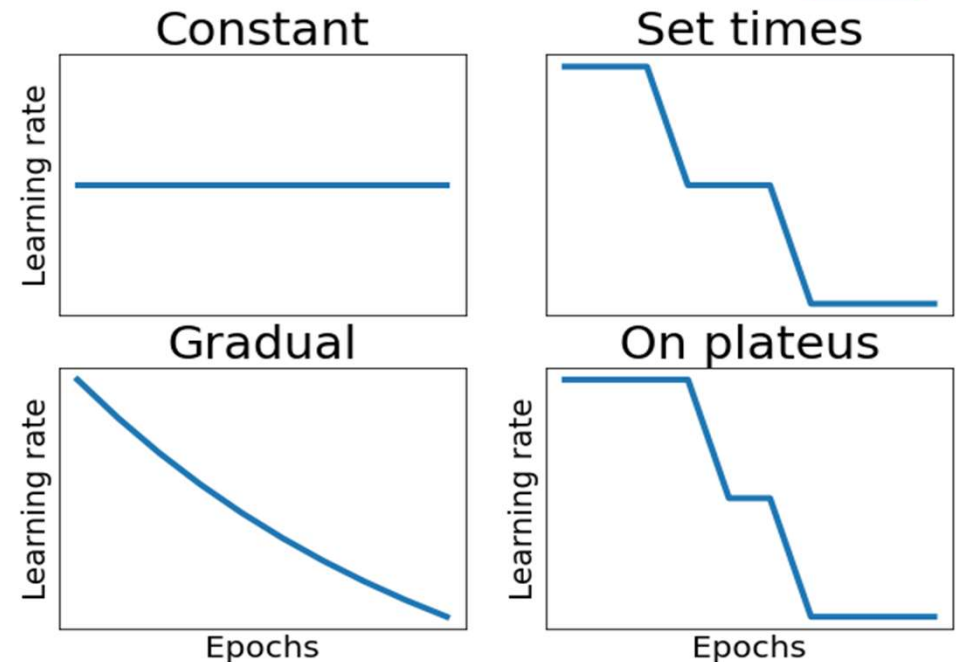
LR schedulers in pytorch

Pytorch has a number of predefined LR schedulers:

<https://pytorch.org/docs/stable/optim.html>

Some of them can even be made to adjust depending on the behavior of the validation loss.

Since 'lr' lives inside the optimizer, the syntax is always that the scheduler takes the optimizer as input.



```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
scheduler = ReduceLROnPlateau(optimizer, 'min')
for epoch in range(10):
    train(...)
    val_loss = validate(...)
    # Note that step should be called after validate()
    scheduler.step(val_loss)
```

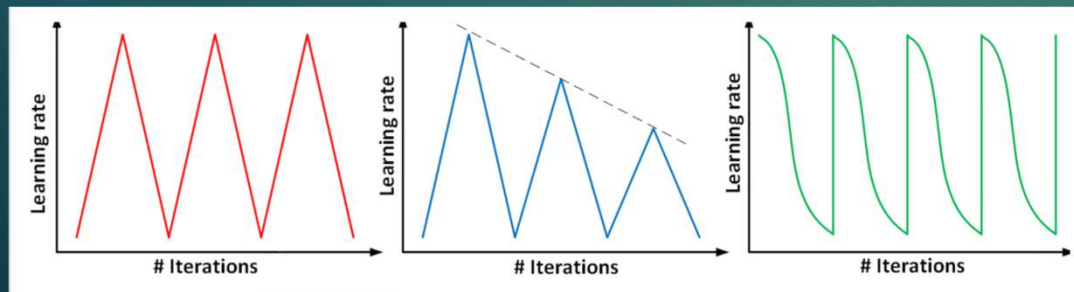
Cyclical schedulers

39

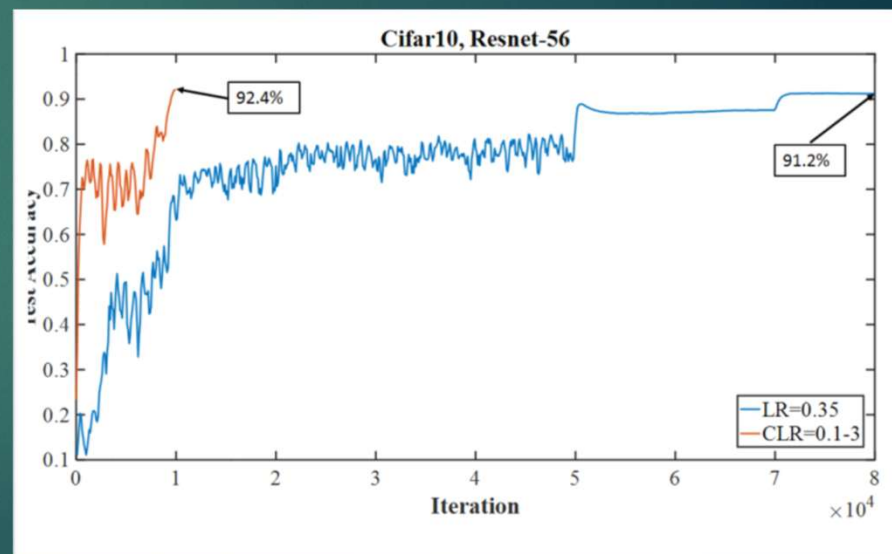
Cyclical learning rate schedulers vary the learning rate between two limits.

Often, in particular the upper limit will be considerably larger than what we would use without a scheduler.

In their 2017 paper, Leslie Smith and Nicholay Topin describe that by choosing a '1-cycle' learning rate schedule, where lr starts small, grows large and decays back, they are able to reach the same high accuracy, but much quicker



They call this phenomenon **Super-convergence**:

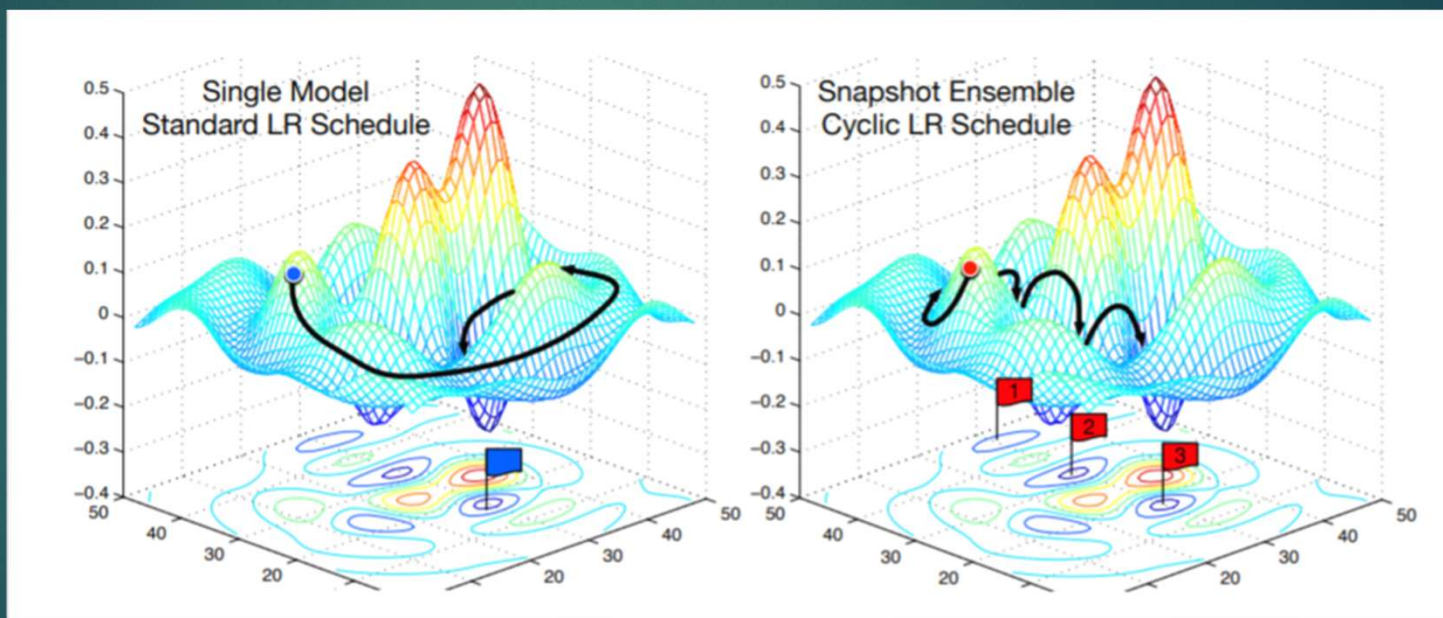


Source: <https://arxiv.org/abs/1708.07120>

Cyclical schedulers

40

On the topic of cyclical schedulers, some researchers have also proposed using multi-cycle schedulers to obtain an ensemble of models from different parts of the parameter space, using only a single training run:



Sources: <https://arxiv.org/pdf/1704.00109.pdf>, <https://arxiv.org/pdf/2003.11266.pdf>

About 'part 2':

41

Someone actually went and wrote Leslie Smith about part 2, this was apparently his response:

Dear Eli,
Thank you for your interest in my research.
When I wrote that report I was planning a second part on architectures but a number of papers that appeared that seemed to cover the topic, so there isn't a part 2.
Two papers I would recommend are:
EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks
at <https://arxiv.org/abs/1905.11946> 109
Designing Network Design Spaces at <https://arxiv.org/abs/2003.13678>

So I guess if you liked part 1, add these to your reading list.

Batch size & Loss

Set 'reduction' to 'mean':

If you are using the (mini)batch loss to guide your hyperparameter search, and one of the hyperparameters is the batch size, remember to set 'reduction' to 'mean' for your loss function. Otherwise your losses won't be comparable for different batch sizes.

```
loss_fn = nn.MSELoss(reduction='Mean')
```

Also:

Often times we are interested not only in the final loss, but also the time it took to get there. Since large batch sizes will generally go through the data in a shorter time, it can be good to use '#calculations' or actual time passed when comparing different batch sizes, instead of number of epochs. Generally, small batch sizes will need fewer epochs to converge.

For a FLOP-counting library, have a look at

https://github.com/facebookresearch/fvcore/blob/main/docs/flop_count.md

My advice on model complexity

When you're just getting started with your training, make the model overfit. You can do this either by:

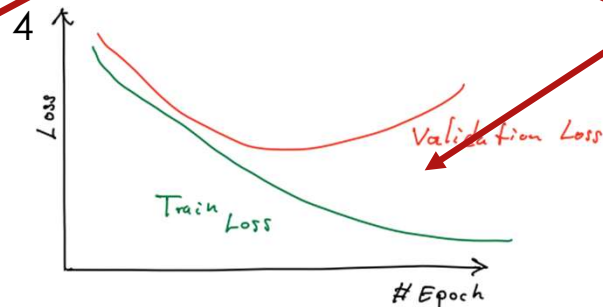
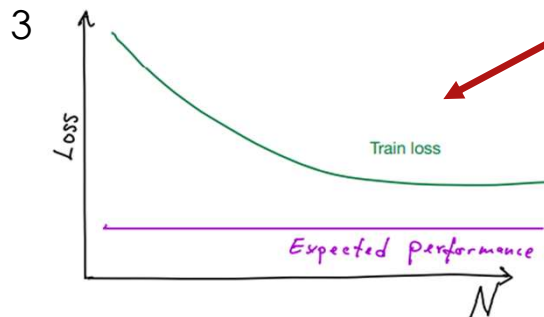
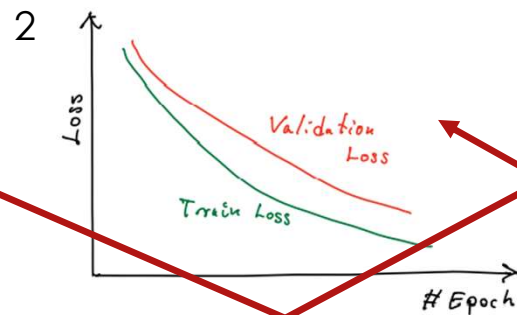
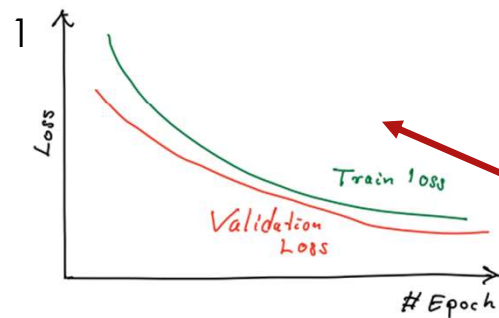
- Using very little data. This has the advantage that it trains really quickly, so you can do your debugging quickly too.
- Using a very large model. This has the advantage that you can, most likely, simply adjust your regularization afterwards.

The point of this is that if you can't overfit in the extreme case, you know there's something fundamentally wrong with either your model or the way you train it.

Once it overfits, start moving the other way (increasing training data or regularization) until validation accuracy begins to climb. Now you know, roughly, what level of complexity your data set can support.

Bias & variance quiz

Below are some examples of loss curves. For each of them, discuss what they might tell us. On the right are some suggestions



44

Too small model

Too much validation data

Too big model

Too much training

Not enough training

Not enough sleep

Not enough validation data

Not enough training data

Too much training data

Summing up again:

45

Quick, think of two things I just covered!

Summing up again:

46

Data splitting

Search strategies

Leslie:

- Map out the range of your variables
- Consider your regularization
- Models that end low tend to drop quickly
- Consider a learning rate scheduler

Loggers & trackers

This search process gets a lot more tractable if you use a dedicated service to track your results.

There are a lot of different ready made solutions out there.

Special mention goes to Neptune, Weights & Biases, mlflow, comet and TensorBoard, but there are many more options out there.



47



Weights & Biases



TensorBoard

<https://neptune.ai/blog/best-ml-experiment-tracking-tools>

Neptune

I have decided to use **Neptune** in my projects, somewhat randomly (I found a lot of well-written blog posts by them when I was looking into loggers...).

Many of the other loggers can do similar things, but I will show you how to get started with Neptune.

48



Colab:

```
!pip install neptune-client

#start neptune run:
import neptune.new as neptune

run = neptune.init(project='mikkelsen',
                  api_token=token) #

[ ]
params = {"learning_rate": 0.001,
          "optimizer": "Adam"}
run["parameters"] = params

for epoch in range(10):
    run["train/loss"].log(0.9 ** epoch)

run["eval/f1_score"] = 0.66

[4] run["config/day"]='tues'
run["config/time"]='now'

[5] run["monitoring/handMade"].log(1)

[7] run["monitoring/handMade"].log(4)
run["monitoring/handMade"].log(3)
run["monitoring/handMade"].log(5)
```

The way to use Neptune is the following:

Before using:

- Register an account
- Make a workspace ('sandbox' in our case)

Using:

- Before starting the run, call `Neptune.init()`
- Set parameter values like

```
run["parameters"] = params
```

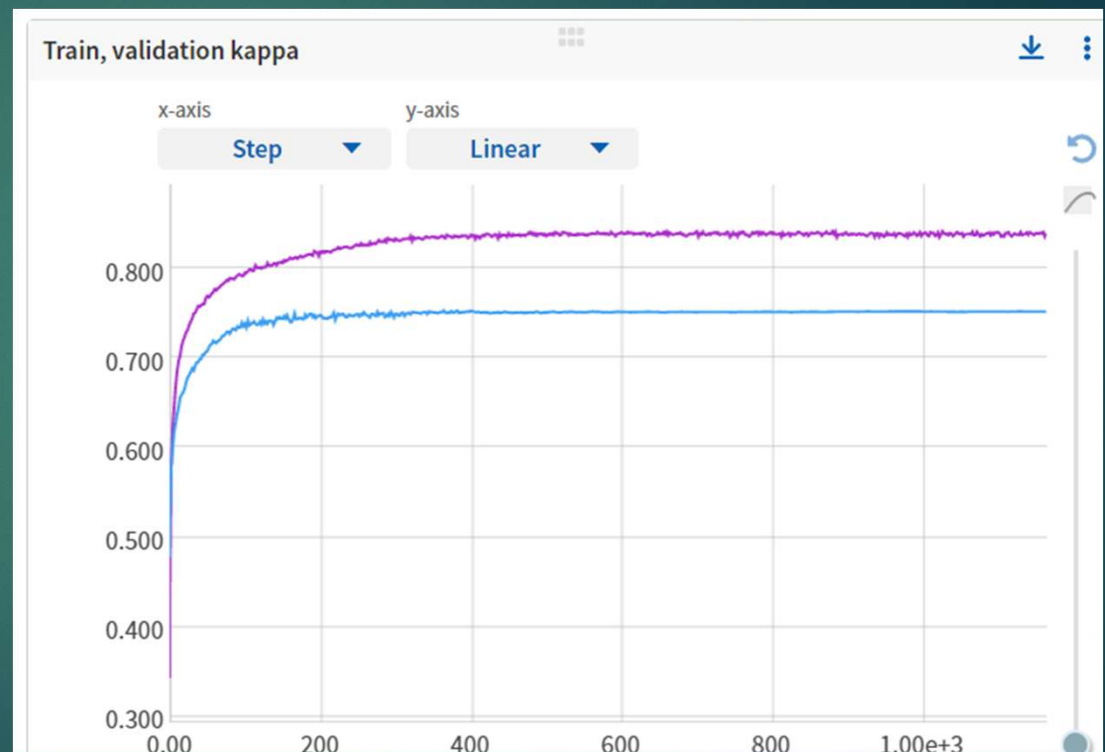
(you can use subfolders to group the parameters)

- Log evolving values (such as losses) using `run.log()`

Other features:

50

Use Neptune to monitor calculations running on remote hardware. Once your jobs have started, you can simply use a browser to check on them.

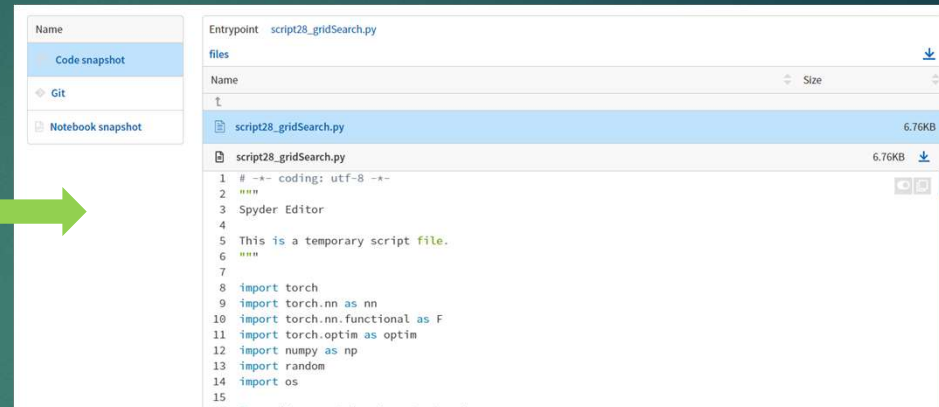


Other features:

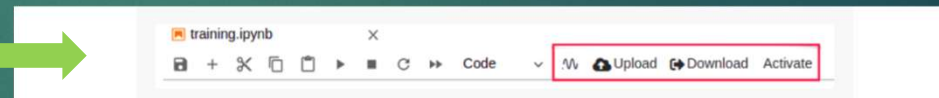
51

Neptune makes version control *much* easier for you.

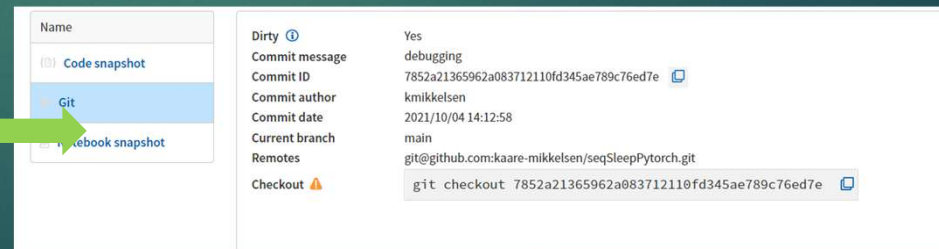
- It can automatically save a copy of the script that started the run.



- Or a snapshot of the notebook



- Or even the state of the git repository, if you started a script inside a repository.

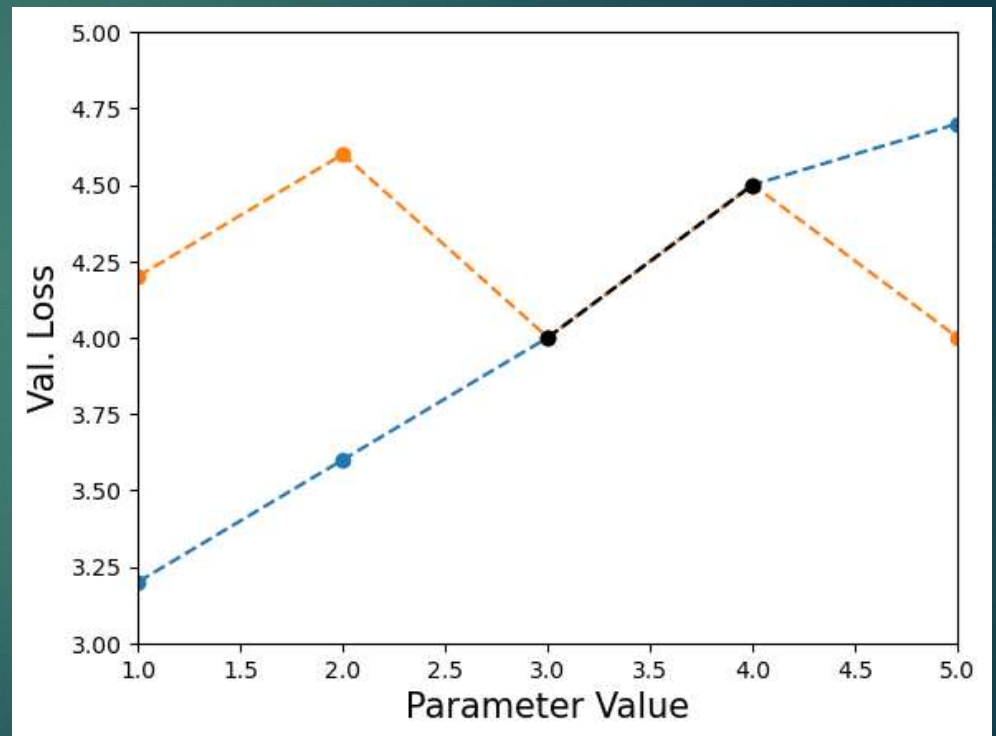


Good advice for ML projects in general:

If you change a parameter –
test more than two values.

5 is usually enough for a plot.

Special tip: do it first
and then ask yourself
if it's just random
noise?



Good advice for ML projects in general:

Which brings me to another reminder:

You can still do statistical testing.

If you're unsure whether some elaborate change to the model / training scheme actually makes a difference, run a bunch with different random seeds, and make a test (either t-test, permutation test, or something more fancy).

I have seen otherwise smart students spend half their thesis period doing elaborate A/B testing to develop their model, but never considering whether their results were significant.

You can end up with some pretty nonsensical results that way.

Network architecture

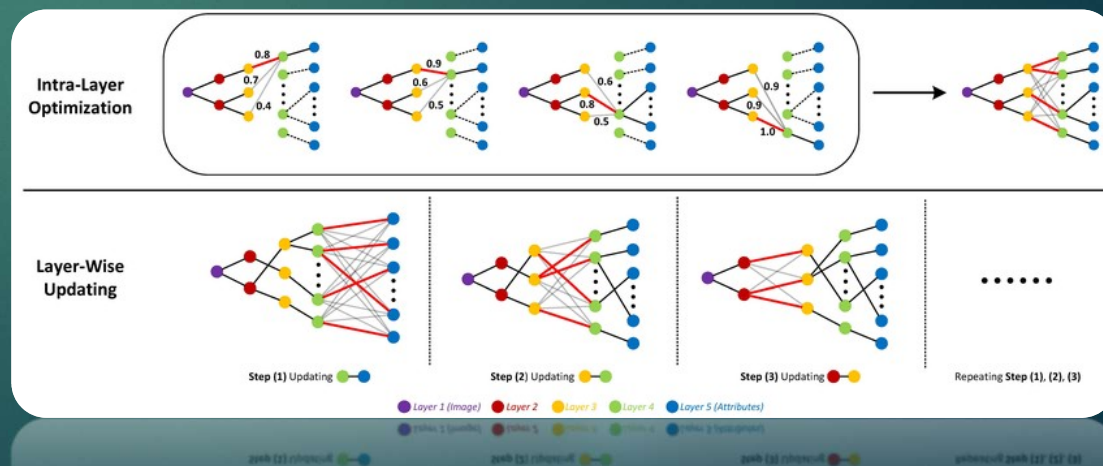
NAS: Neural Architecture Search

55

Like we can search for the optimal values of lr, batch size, weight decay etc., we can also imagine a methodical search for the optimal network architecture.

A landmark study of this was Zoph & Le 2017, in which reinforcement learning was used to train a separate 'controller' network, which then spawned a very large number of 'child' networks.

However, this was incredibly slow. The first study apparently used 800 GPU's for 28 days...



Source: <https://arxiv.org/abs/1611.01578>

NAS: Neural Architecture Search

Since then, various tricks have been proposed to speed up the search.

Training speed-up:

- Using less data
- Extrapolating loss curve

Weight sharing:

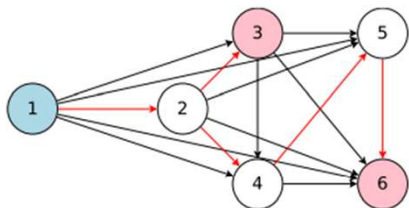
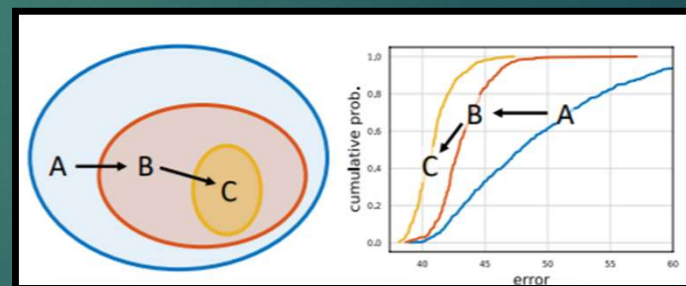


Figure 2. The graph represents the entire search space while the red arrows define a model in the search space, which is decided by a controller. Here, node 1 is the input to the model whereas nodes 3 and 6 are the model's outputs.

Reduced search space:

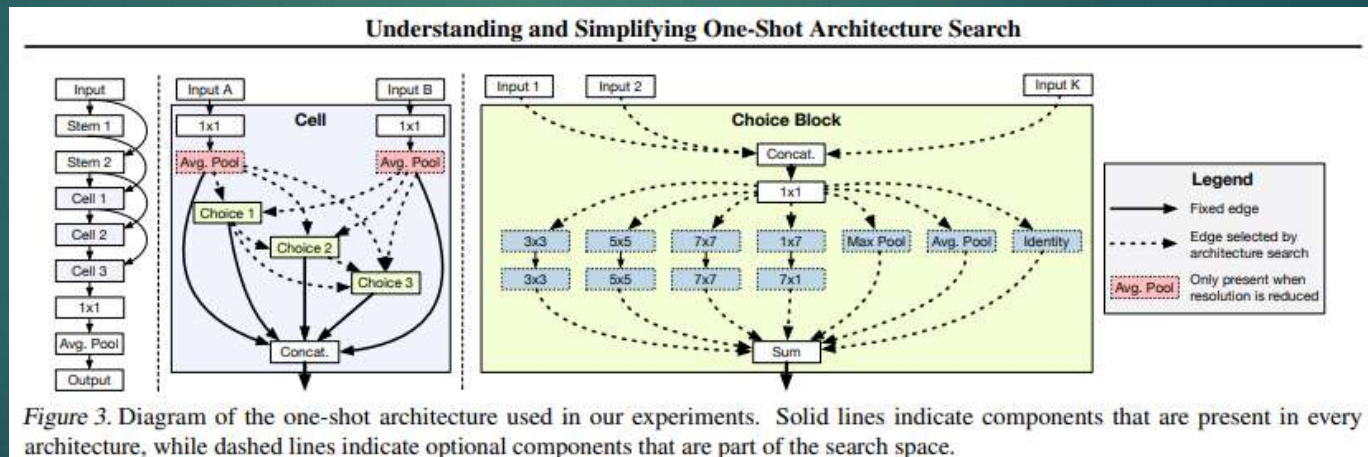


One Shot NAS

57

Weight sharing is also called 'one shot' NAS.

In this, the entire search space is represented by one large model, with alternating paths.

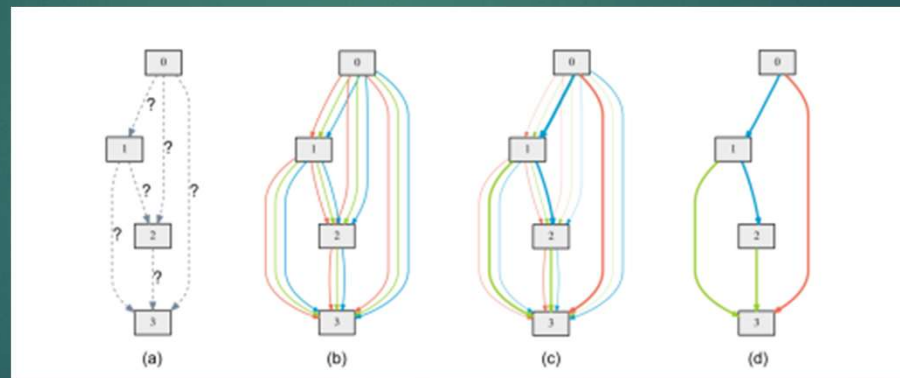


<https://proceedings.mlr.press/v80/bender18a/bender18a.pdf>

DARTS

58

'Differentiable Architecture Search' is a popular 'One Shot' approach. Here, the weights are differentiable, and the model 'anneals' into the optimal architecture through backpropagation:



[DARTS: 1806.09055v2](#)

Reading more

NAS is a very active field of research.

You can try to keep up to date using this handy list of NAS papers:

<https://www.automl.org/automl/literature-on-neural-architecture-search/>

(more than 1000 papers has been published since beginning of 2023)

The authors of the list recently published this quick NAS intro:

"Bag of Tricks for Neural Architecture Search": <https://arxiv.org/pdf/2107.03719>

Preparing this lecture, I also came across this torch-based toolbox:

[NNI Documentation — Neural Network Intelligence](#)

Skipping the search

60

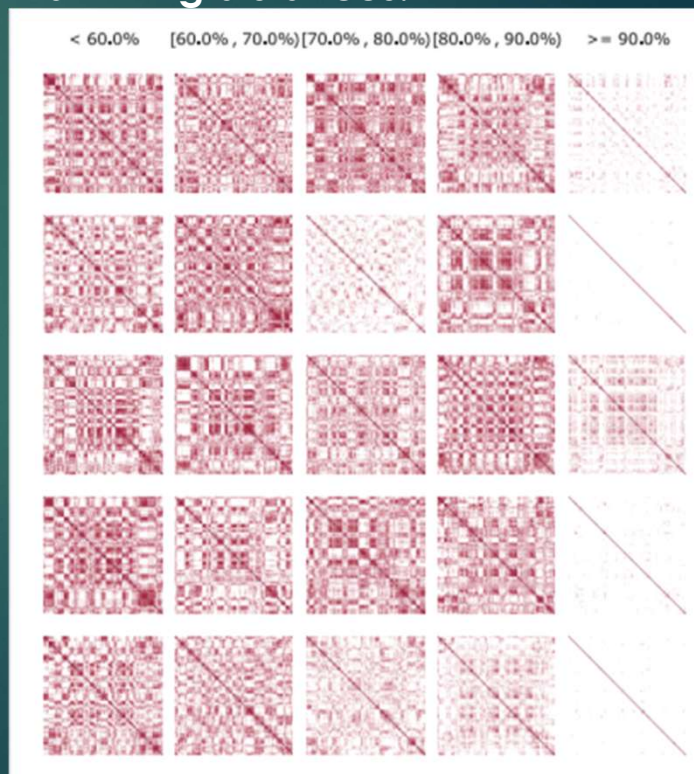
An intriguing idea is to predict the performance of a given network design without any training at all. "*Neural Architecture Search without Training*" is a proposal for doing just that.

In their study, Mellor et al looked at the activations of ReLU neurons to all data points in a minibatch. For each datapoint, the activations formed a bit string ('binary code').

Using the Hamming distance, it is possible to quantify the distance between two bitstrings.

The authors found that for good networks, the Hamming distance for two data points is generally high. Meaning that good network architectures will have very different activations for different inputs.

Hamming distances:



Source: <https://arxiv.org/abs/2006.04647>

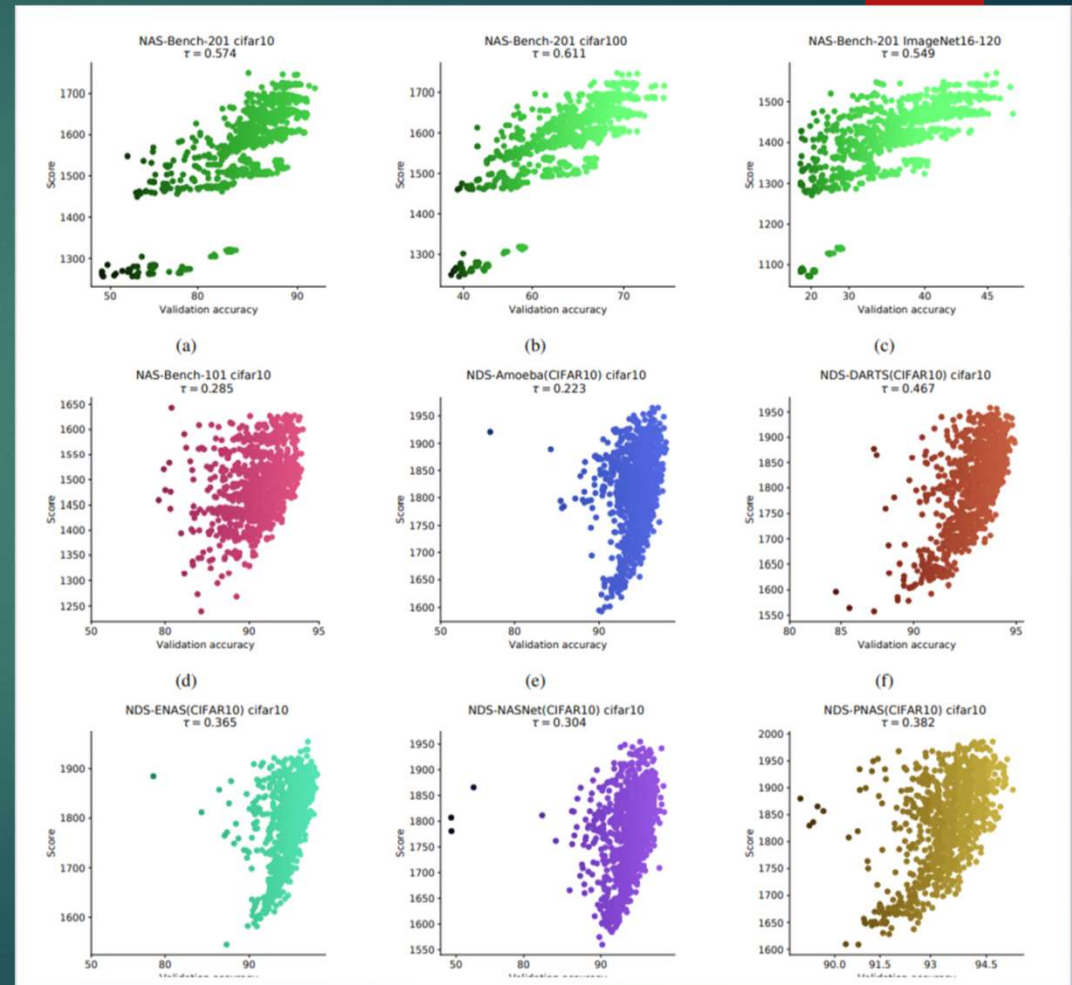
Skipping the search

61

By calculating the norm of the matrix of Hamming distances, the authors obtained a score for each network architecture:

$$s = \log |K_H|$$

In the plots to the right, we see that there is some general trend, but for a given score (y-axis) a lot of different validation accuracies are possible.



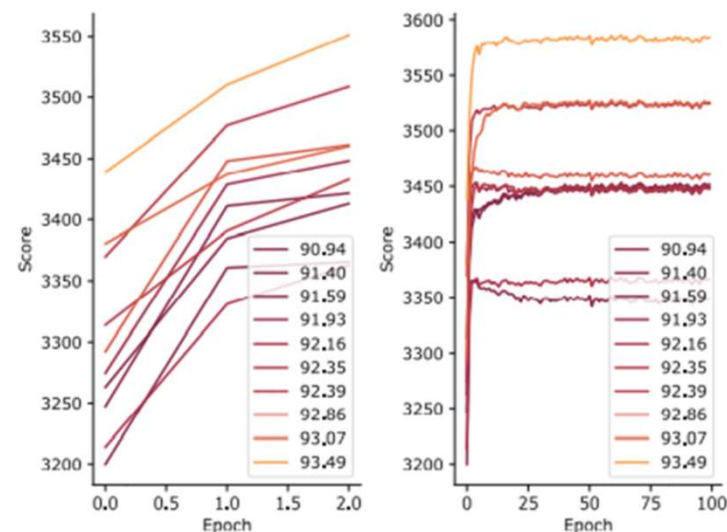
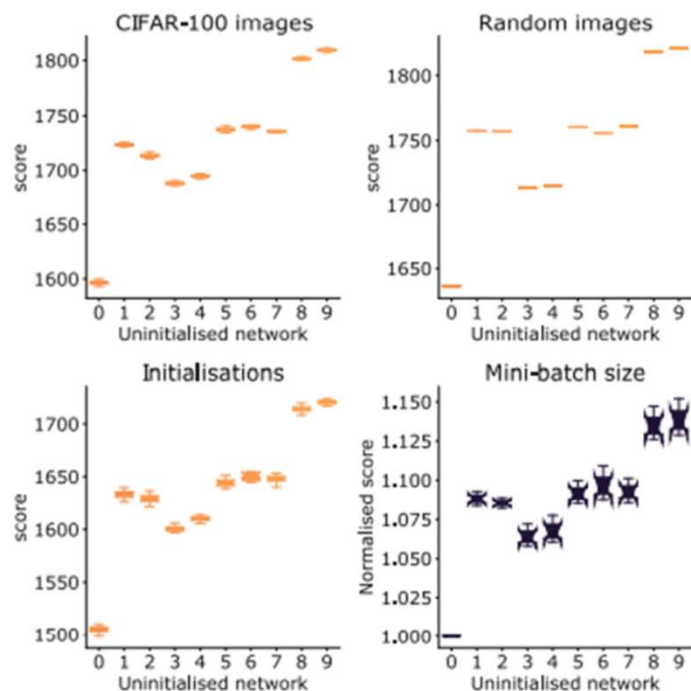
Source: <https://arxiv.org/abs/2006.04647>

Skipping the search

62

Interestingly, it seems that the score values are more or less independent from initialisations and batch sizes.

Additionally, the score of an individual network seems to change very little during training.



More on this topic:

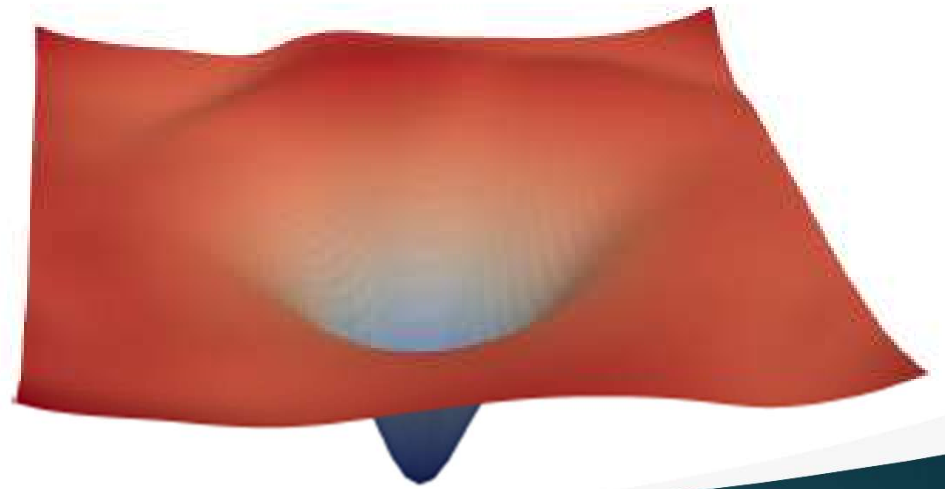
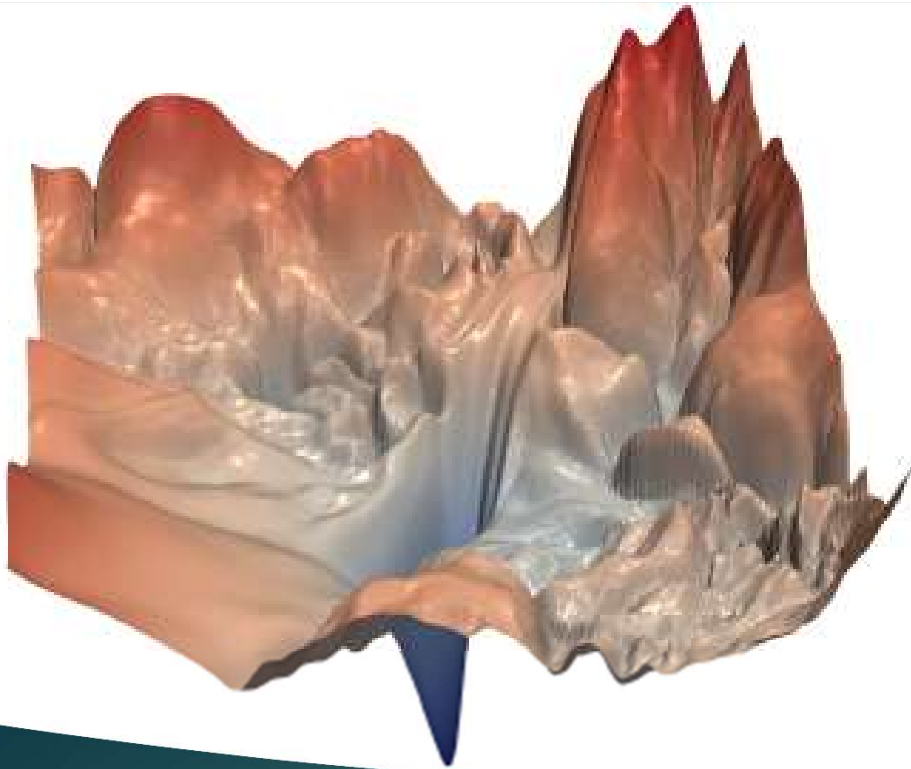
63

Given its potential, this topic is still receiving attention, and a review paper can be found here:

" Training-free neural architecture search: A review", Wu & Tsay 2023.

<https://www.sciencedirect.com/science/article/pii/S2405959523001443>

The primary take away is that while the approach is exciting, it's not quite there yet...

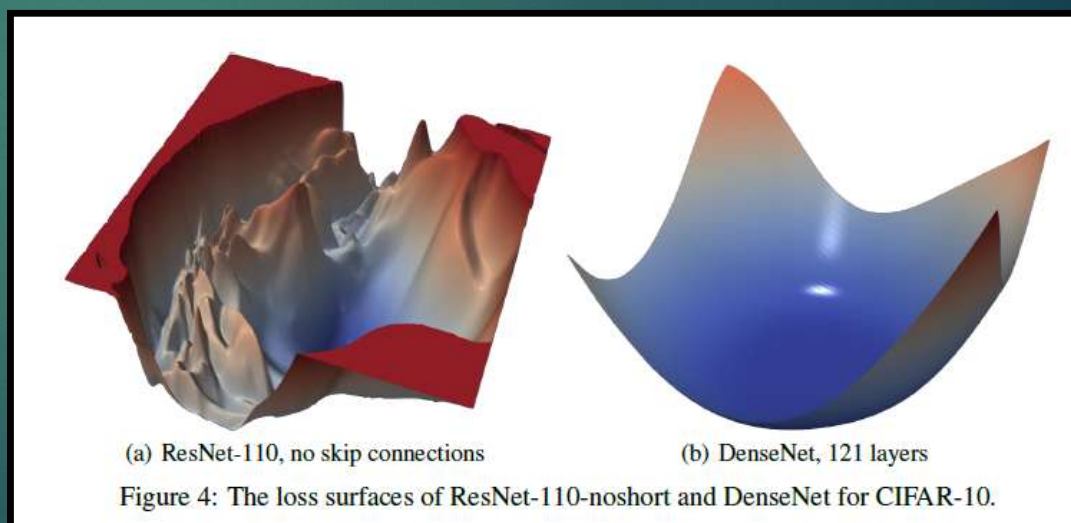


Loss landscapes

Loss Landscapes

Not only is the size and shape of the model important to the loss that can be achieved, but it turns out that some models are simply easier and quicker to train. We may qualitatively explain this by the 'shape' of the loss landscape.

In '*Visualizing the Loss Landscape of Neural Nets*', Li et al set about trying to visualize the high dimensional loss landscapes.



Source: <https://arxiv.org/abs/1712.09913>

Filter-Wise Normalization

A central feature of loss landscapes is their 'flatness'. To be able to compare that between visualizations, length scales become important. The authors propose a way to normalize directions in weight space.

Let M be a trained network, let θ be its weights, and let d be a random, Gaussian vector in weight-space.

We wish to use d as perturbation on θ , but first we scale it:

$$d_{ij} \rightarrow |\theta_{ij}| \frac{d_{ij}}{|d_{ij}|}, |\cdot|: \text{Frobenius norm}$$

Here, d_{ij} is the i 'th filter in the j 'th layer.

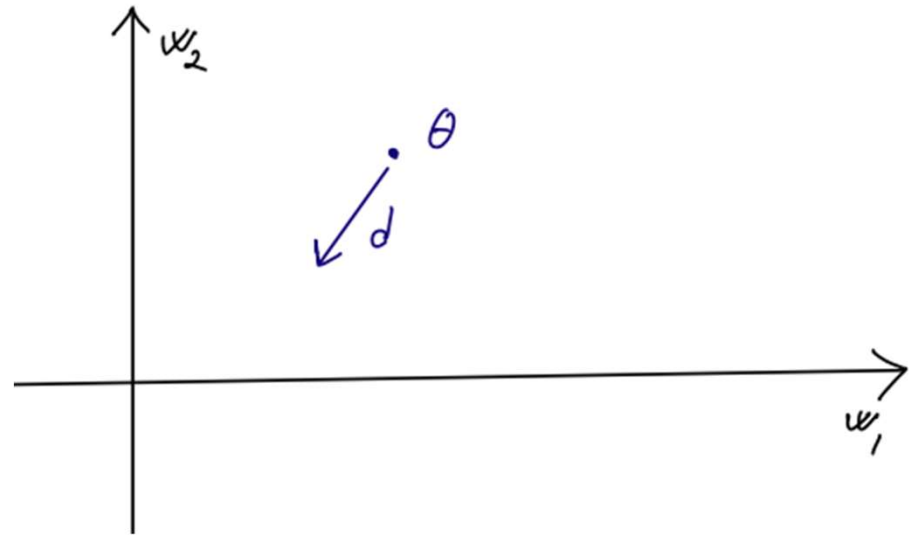
The motivation is that since d will act as a direction of perturbation to θ , it is important that these perturbations have the same relative magnitude for each part of the network.

This interpretation implicitly assumes ReLU activations and networks consisting only of convolutional and fully connected layers.

Filter-Wise Normalization

In other words, we are considering perturbations around a local minimum in weight-space.

Perturbations are unit vectors multiplied by the size of Θ .



Sanity checks

The authors are interested in explaining why some models generalize better than others (meaning smaller difference between test and train loss).

They find that after using filter-wise normalization, and plotting the loss along a random direction through the minimum, that wide minima are associated with lower generalization error.

Which seems sensible enough.

68

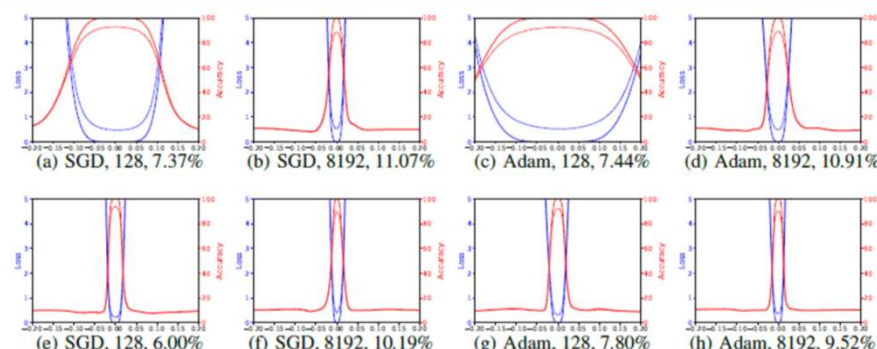


Figure 12: Enlarged Figure 11. The range of the x-axis is $[-0.2, 0.2]$ instead of $[-1.0, 1.0]$. The first row has no weight decay and the second row uses weight decay 0.0005. The pairs (a, e) and (c, g) show that sharpness of minima does not correlate well with test error.

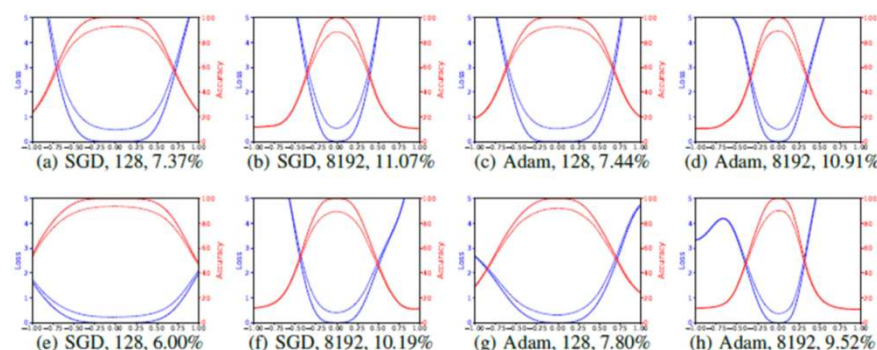
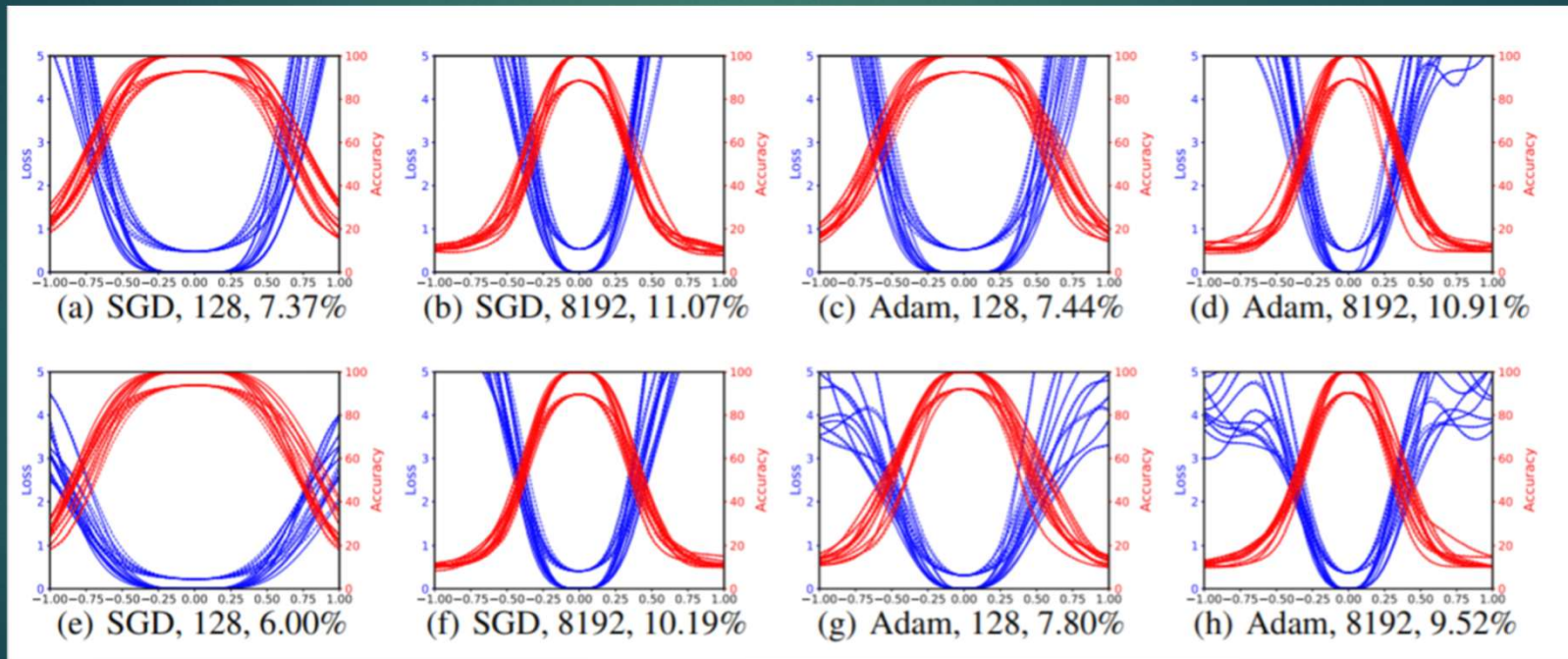


Figure 13: 1D loss plots for VGG-9 with layer normalization. The first row has no weight decay and the second row uses weight decay $5e-4$.

Sanity checks

69

Interestingly, it seems that this is robust to different d-directions:



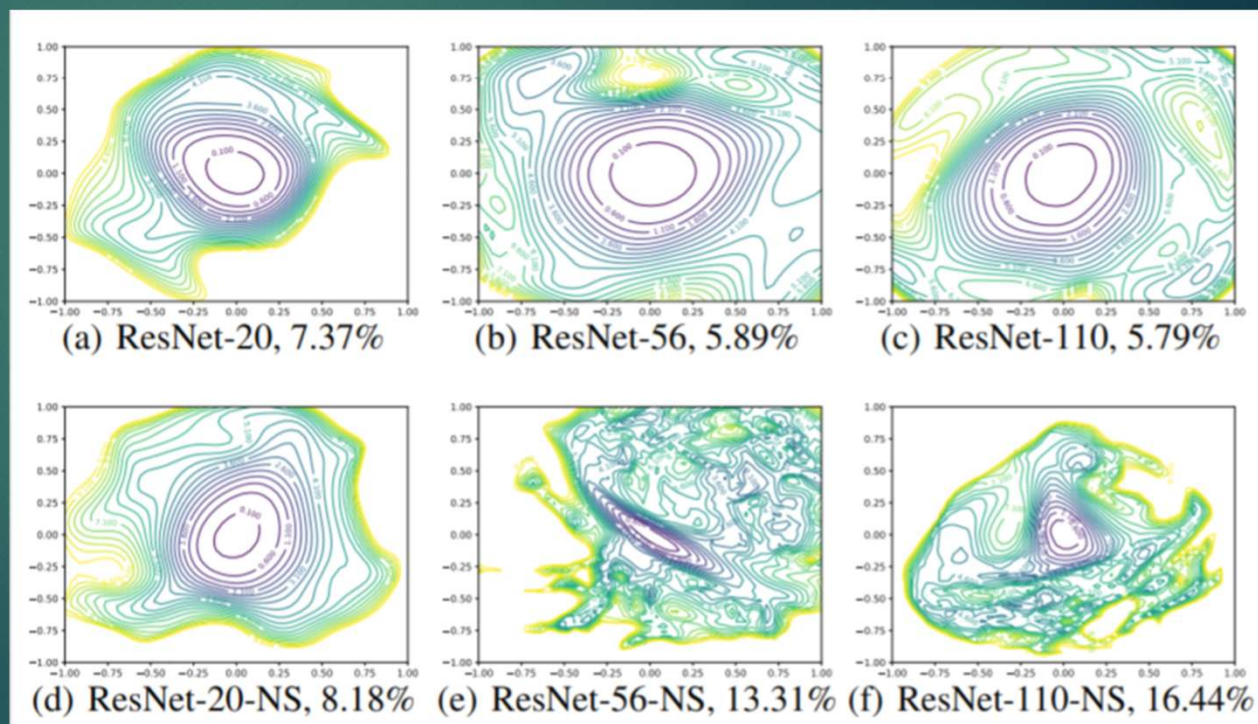
2D plots

70

Since apparently the behaviour is preserved no matter the direction chosen, we could pick two random, orthogonal directions, and calculate the loss function for that plane.

This gives us landscapes, and it seems that the narrow minima are associated with more chaotic landscapes.

The authors mention that the chaotic landscapes are also associated with networks that do not converge for unfortunate initial conditions.



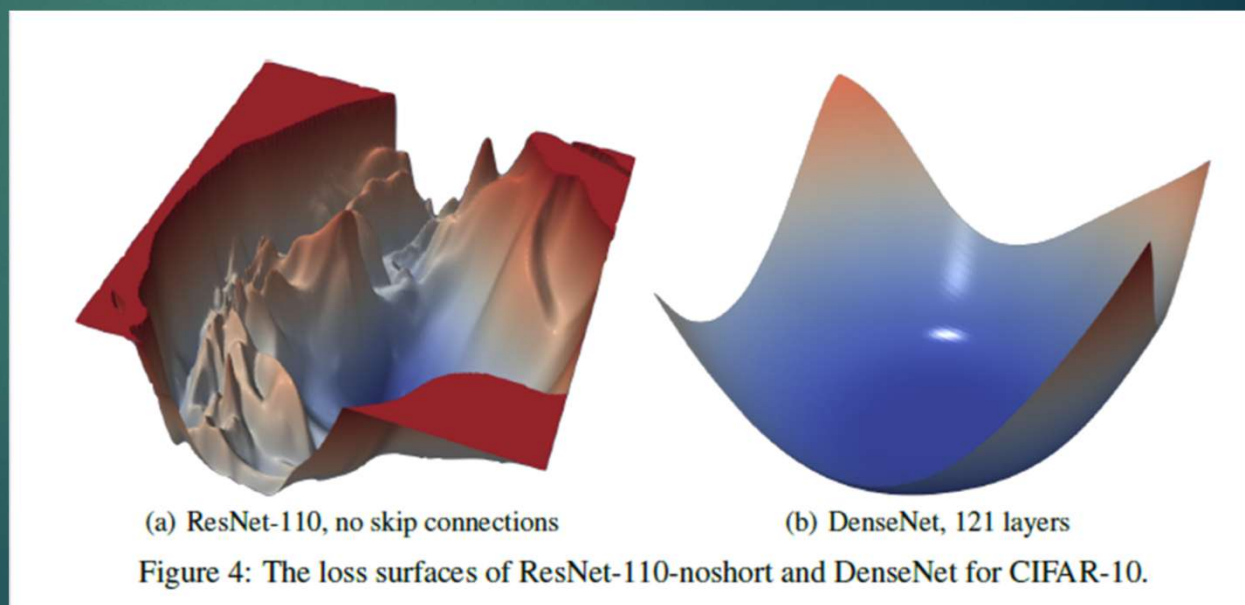
Skip Connections

71

It has previously been observed that skip connections (residual layers) drastically improve the trainability of a network.

The authors look into this, and find, again, that networks with skip connections seem to have much smoother landscapes close to the minimum.

We should probably consider this as separate benefits, not directly connected with those we have previously discussed (more direct gradient paths, having the identity in the space of possible functions).



Hessian

As a final reality check, the authors calculate the eigenvalues of the Hessian in all positions of the loss landscape.

In the plots below, the relationship between the absolute value of the most negative eigenvalue vs the most positive eigenvalue is plotted. We see that the convex regions from before correspond to regions without eigenvalues much smaller than 0.

In other words, it seems that the easy to train models are easy in most directions.

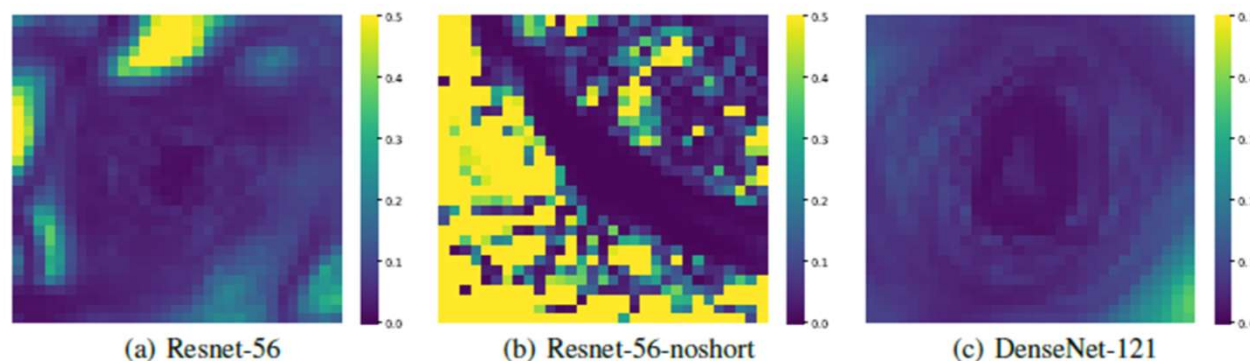


Figure 7: For each point in the filter-normalized surface plots, we calculate the maximum and minimum eigenvalue of the Hessian, and map the ratio of these two.



'Træerne vokser ikke
ind i himlen'

Limitations

73

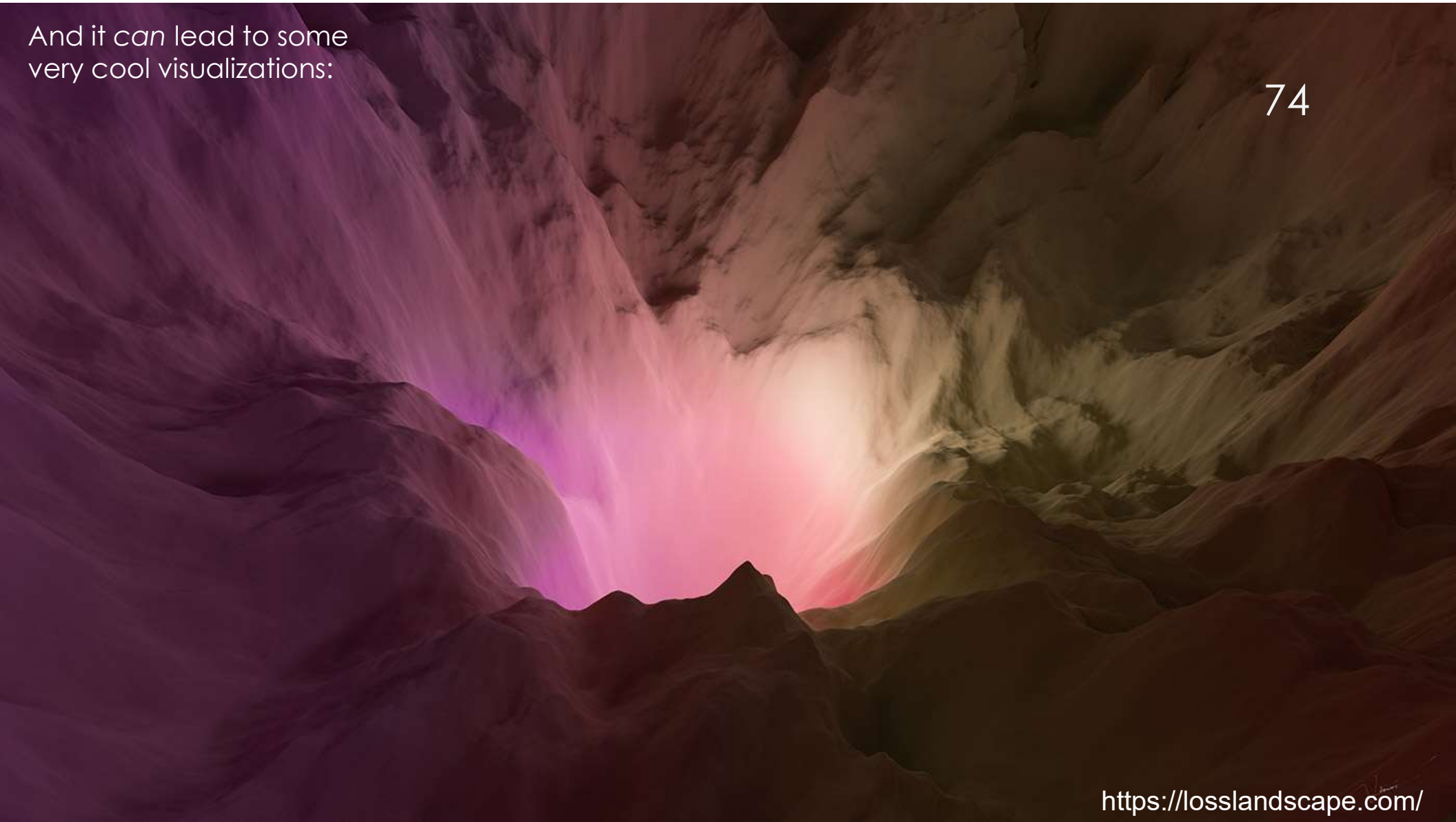
- The investigations of this phenomenon are not quite as systematic as one could hope. The authors do perform some tests, but the overarching theme of the paper still feels anecdotal.
- These visualizations do not exactly describe why skip-connections or small batch sizes can give better generalizations - they just give us another way to describe that.

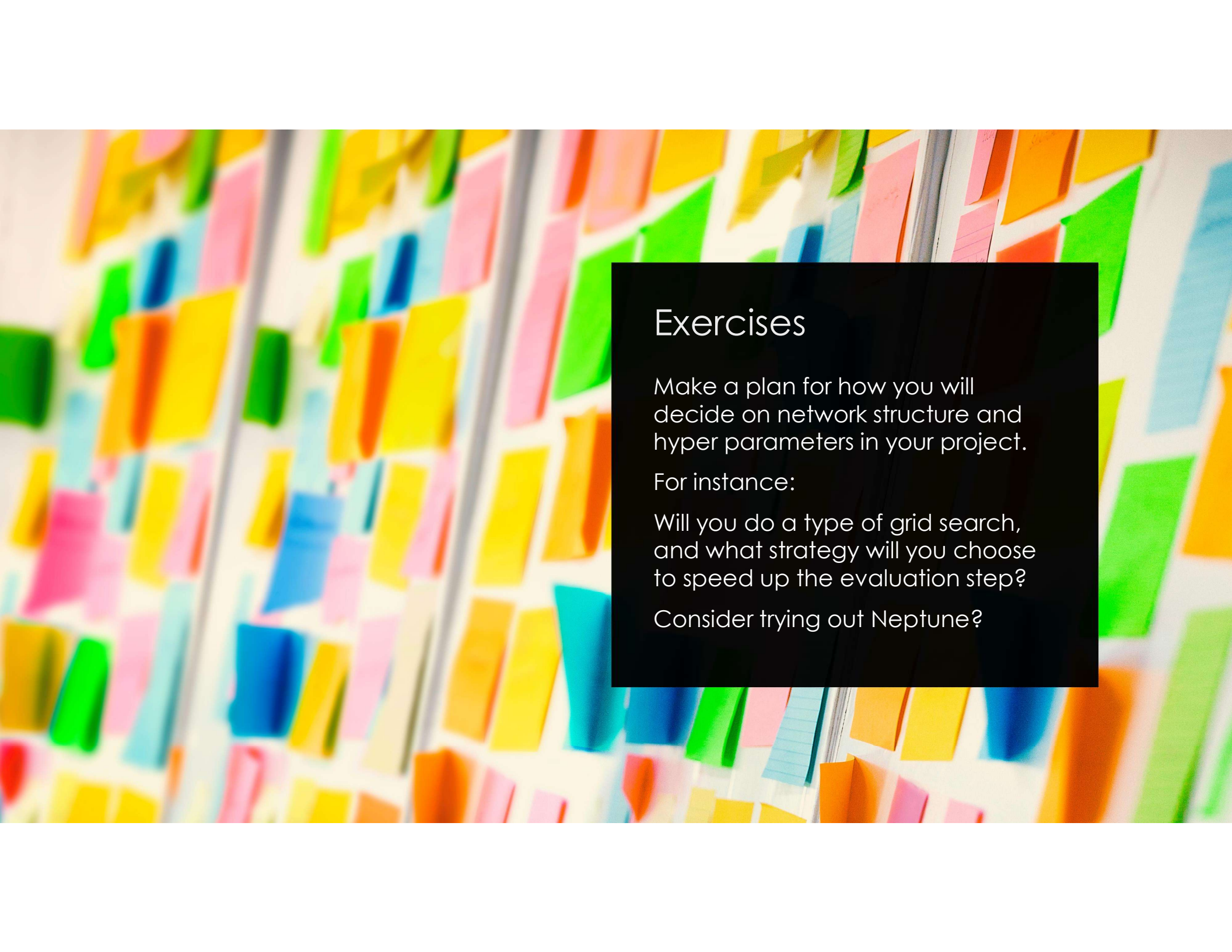
However, the plots are intriguing, and I must admit that they have inspired some new ideas for me 😊

And it *can* lead to some
very cool visualizations:

74

<https://losslandscape.com/>





Exercises

Make a plan for how you will decide on network structure and hyper parameters in your project.

For instance:

Will you do a type of grid search, and what strategy will you choose to speed up the evaluation step?

Consider trying out Neptune?

Reading & watching

“A disciplined approach to neural network hyper-parameters: Part 1 -- learning rate, batch size, momentum, and weight decay”:

<https://arxiv.org/abs/1803.09820>

What Andrew Ng has to say about "Debugging ML Models and Error Analysis":

<https://www.youtube.com/watch?v=ORrStCArmP4>